SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI-602105

NAME	AKSHAYA R S
COURSE CODE	CSA0209
REG NO	1924260209
GITHUB LINK	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING-1924260209

Grama Jyothi Rural Electricity Co-operative: Billing Program

TASK 1 - UNDERSTAND THE PROBLEM AND IDENTIFY THE DATA

1. Aim

To study the billing problem and list every data item the program must store or calculate. For each item I decide whether it is an input, an output or a constant, and I give a reason. I also state the rules every correct bill must follow and the assumptions I make.

2. Explanation and Analysis

2.1 What went wrong in the spreadsheet

Fault	Cause	Design lesson
Bills off by a few paise	Rupees stored as fractions (e.g. 2.25 × units) and rounded in different places	Store money as whole paise
Negative bill after 65535 → 0	Meter is a 16-bit counter. Current (e.g. 120) minus previous (e.g. 65400) is negative	Handle roll-over with unsigned 16-bit arithmetic
Tariff revision applied partly	The tariff was typed in many cells	Define each tariff value once

2.2 Inputs, outputs and constants

#	Data item	Kind	Purpose	Justification
1	Household number	Input	Identifies the bill	A whole number up to 800,000, which does not fit in 16 bits. Never negative, so unsigned
2	Category (e.g. 'D' domestic)	Input	Allows different tariffs later	A single letter, stored as a character
3	Previous reading	Input	Meter value last month	Whole number 0 to 65535 (16-bit register)
4	Current reading	Input	Meter value this month	Same range as the previous reading
5	Units consumed	Output / intermediate	current minus previous (with roll-over)	Never negative. At most 65535
6	Energy charge	Output	Money for units, slab by slab	Fractional in rupees (Rs 2.25) but a whole number in paise
7	Fixed charge	Output (from constant)	Charge every household pays	Constant Rs 50 = 5000 paise
8	Total bill	Output	Fixed charge + energy charge	Whole paise
9	Co-operative total	Output	Sum of all bills	Largest value in the program (district scale)

Constants (never change during a run):

- Slab limits: 100 units and 200 units.
- Rates: 0, 225 and 450 paise per unit.
- Fixed charge: 5000 paise.
- Meter maximum: 65535, so the roll-over modulus is 65536.
- Paise per rupee: 100.

Each of these is used in many places. Defining it once is the fix for the "tariff revision applied partly" fault. **2.3 Which values can be negative, fractional or very large**

Property	Which data	Explanation
Can be negative	The raw difference current − previous	Negative after roll-over. It must be corrected and never printed as a bill
Never negative	Units, energy charge, total bill	A valid bill is at least Rs 50

Fractional	Rates (Rs 2.25, Rs 4.50)	Converting to paise (225, 450) removes the fraction
Very large	Total of all bills	Calculated below

Largest values (calculation):

- Largest units in one month = 65535.
- Largest bill = 5000 + (100 × 0) + (100 × 225) + (65535 − 200) × 450 = 5000 + 22500 + 65335 × 450 = 5000 + 22500 + 29,400,750 = **29,428,250 paise = Rs 294,282.50**.
- A typical household uses about 150 units, so a bill is about 16,250 paise (Rs 162.50).

Scale	Bills	Typical total (150 units each)	Fits in 32-bit int (max 2,147,483,647)?
Village	4,000	4,000 × 16,250 = 65,000,000	Yes
Taluk	60,000	60,000 × 16,250 = 975,000,000	Yes, but close (45%)
District	800,000	800,000 × 16,250 = 13,000,000,000	**No**, about 6 times too large

The worst case at district scale is 800,000 × 29,428,250 = about 2.35×10^{13} paise. This needs a 64-bit type (see Task 2).

2.4 Rules every correct bill must satisfy

1. Units consumed is at least 0.
2. Units consumed is at most 65535, because the meter can roll over at most once in a month.
3. Energy charge is at least 0 and total bill is at least the fixed charge (Rs 50).
4. **Total = fixed charge + energy charge** exactly, with no rounding difference.
5. The bill never decreases when units increase (monotonic).
6. Every amount is a whole number of paise.
7. The sum of all individual bills equals the co-operative total.
8. Every tariff value comes from one place only.

2.5 Assumptions

1. Readings are whole units (meters show integers).
2. A household uses fewer than 65536 units in one month, so at most one roll-over occurs.
3. current equal to previous means 0 units used (not a full 65536-unit cycle).
4. One tariff applies to all categories for now. The category field is stored so that different tariffs can be added later.
5. The slab rule is: first 100 units free, units 101 to 200 at Rs 2.25, units above 200 at Rs 4.50. The fixed charge of Rs 50 is charged even for 0 units.
6. No taxes, discounts or late fees are included.
7. Household numbers run from 1 to 800,000.

3. C Program / Code

Task 1 is analysis only. A small program is given below for the alternative concept.

4. Alternative Concept: readings stored as text

4.1 What the solution looks like

Readings are stored as strings such as "01530" (always 5 digits, zero-padded). Before the bill can be calculated, the strings must be converted to numbers digit by digit.

```
#include <stdio.h>

/* Convert a 5-digit text reading to an integer: O(d) */
unsigned int text_to_number(const char *s)
{
    unsigned int value = 0;     for
(int i = 0; s[i] != '\0'; i++) {
value = value * 10 + (unsigned)(s[i] -
'0');   /* one digit at a time */
    }
    return value;
}
```

```
int main(void)
{
    char prev[] = "01200";
char curr[] = "01530";
    unsigned int units = text_to_number(curr) - text_to_number(prev);
printf("Units = %u\n", units);
    printf("As text: %zu bytes, as unsigned short: %zu
bytes\n",            sizeof curr, sizeof(unsigned short));
return 0; }
```

Sample output (I ran the size line separately and got 6 and 2; verify the whole program with GCC):

```
Units = 330
As text: 6 bytes, as unsigned short: 2 bytes
```

4.2 Complexity analysis (d = number of digits, here 5)

Operation	Integer representation	Text representation
Read one reading	O(1) (%hu converts it once)	Stored directly, then converted in O(d) later
Convert to number	Not needed	O(d) (one multiply-add per digit)
Compare two readings	O(1)	O(d) with strcmp (same length, zero-padded)
Subtract	O(1)	O(d) after conversion
Space per reading	2 bytes	d + 1 = 6 bytes (digits + '\0')
Space for 800,000 houses × 2 readings	3.2 MB	9.6 MB

Since d is fixed at 5, the text version is still O(1) in n, but it does about 5 times more work per reading and uses 3 times more memory. Its advantage is that leading zeros are preserved and the digits are easy to display.

5. Evidence / Result

The tables above are the evidence: the data items, the constants, the largest values, the rules and the assumptions. Add a screenshot of the size comparison output.

TASK 2 - CHOOSE THE DATA TYPES

1. Aim

To choose a C data type for every data item of Task 1 and to check its size and range using `sizeof`, `limits.h` and `float.h`. I also show why money must not be stored as `float` rupees and explain 16-bit meter roll-over.

2. Explanation and Analysis

2.1 Sizes and ranges on my system (output of the program in section 3.2; 64-bit Linux GCC)

Type	sizeof (bytes)	Range	C-standard minimum
char	1	−128 to 127	8 bits
short	2	−32,768 to 32,767	16 bits
unsigned short	2	0 to 65,535	16 bits
int	4	−2,147,483,648 to 2,147,483,647	16 bits
unsigned int	4	0 to 4,294,967,295	16 bits
long	8	-9.22×10^{18} to 9.22×10^{18}	32 bits
long long	8	-9.22×10^{18} to 9.22×10^{18}	64 bits
unsigned long long	8	0 to 18,446,744,073,709,551,615	64 bits
float	4	up to 3.4×10^{38}, about 6 digits (FLT_DIG)	-
double	8	up to 1.8×10^{308}, about 15 digits (DBL_DIG)	-

On Windows MinGW, `long` is 4 bytes, so I use `long long` for district totals so that the program works everywhere.

2.2 Chosen type for every data item

Data item	Type	Size	Range needed	Sufficient for district?
Household number	unsigned int	4	1 to 800,000	Yes. unsigned short (max 65,535) is too small
Category	char	1	one letter	Yes
Previous / current reading	meter_t (unsigned short)	2	0 to 65,535	Yes. It matches the hardware register
Units consumed	unsigned int	4	0 to 65,535	Yes
Energy charge, fixed charge, one bill	long (paise)	4 or more	up to 29,428,250	Yes, even if long is only 32 bits (max 2.1×10^9)
Total of all bills	long long (paise)	8	up to about 2.35×10^{13}	Yes (max 9.2×10^{18})

2.3 Why money must not be stored as float rupees

- Computers store `float` and `double` in binary. Most decimal fractions such as 0.1 or 162.35 have no exact binary form. They are stored as the nearest approximation.
- A `float` keeps only about 6 to 7 significant digits. Once a total reaches a few million rupees, adding Rs 162.35 gets rounded to the nearest representable value.
- The **alternative** is to store paise in an integer. Rs 162.35 becomes 16235. Integer addition is exact, so the total is always correct. Rupees and paise are separated only when printing (`p / 100` and `p % 100`).

2.4 What happens to a 16-bit unsigned register after 65535

A 16-bit unsigned register holds 2^{16} = 65,536 values (0 to 65535). After 65535 the next tick wraps to 0. This is arithmetic modulo 65536. The meter value is always (true count) mod 65536.

2.5 Example where float gives a wrong total (my own measurement) Each of n households has a bill of Rs 162.35. The correct total is n × 162.35.

n (bills)	Correct (integer paise)	float total	float error	double total	double error
4,000	Rs 649,400.00	Rs 649,405.00	**+Rs 5.00**	649,400.000000	−0.00000005
60,000	Rs 9,741,000.00	Rs 9,740,885.00	**−Rs 115.00**	9,740,999.999989	−0.000011
800,000	Rs 129,880,000.00	Rs 129,253,280.00	**−Rs 626,720.00**	129,879,999.998130	−0.00187

For 4,000 bills the float total is wrong by Rs 5.00 (500 paise). The integer-paise total (16235 × 4000 = 64,940,000 paise = Rs 649,400.00) is exact. My choice avoids the problem because every addition of whole numbers is exact and no rounding ever happens.

3. C Program / Code

3.1 Complete the Code: TODO 1 (original program preserved, only the TODO body replaced)

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    // TODO 1: Return the units consumed, correct even when the register has rolled over
    // (current < previous); use unsigned 16-bit arithmetic instead of an if-
else     return (meter_t)(current - previous); }

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));   /* rolled over
*/     return 0; }
```

3.2 Program to print sizes and ranges, and the float vs paise experiment

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("char  %zu bytes  %d..%d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
printf("short %zu bytes  %d..%d\n", sizeof(short), SHRT_MIN, SHRT_MAX);
printf("ushort %zu bytes 0..%u\n", sizeof(unsigned short), (unsigned)USHRT_MAX);
printf("int   %zu bytes  %d..%d\n", sizeof(int), INT_MIN, INT_MAX);
printf("uint  %zu bytes  0..%u\n", sizeof(unsigned), UINT_MAX);     printf("long
%zu bytes  %ld..%ld\n", sizeof(long), LONG_MIN, LONG_MAX);     printf("llong %zu
bytes  %lld..%lld\n", sizeof(long long), LLONG_MIN, LLONG_MAX);     printf("ullong
%zu bytes 0..%llu\n", sizeof(unsigned long long), ULLONG_MAX);     printf("float
%zu bytes max %e epsilon %e digits %d\n",            sizeof(float), FLT_MAX,
FLT_EPSILON, FLT_DIG);     printf("double %zu bytes max %e epsilon %e digits %d\n",
sizeof(double), DBL_MAX, DBL_EPSILON, DBL_DIG);

    int sizes[3] = {4000, 60000,
800000};     for (int k = 0; k < 3; k++)
{         int n = sizes[k];
float f = 0.0f;         double d = 0.0;
long long paise = 0;         for (int i
= 0; i < n; i++) {
            f += 162.35f;        /* float rupees
*/             d += 162.35;         /* double
rupees */             paise += 16235LL;    /*
integer paise */
        }
        printf("n=%d float=%.4f double=%.6f paise=%lld.%02lld\n",
n, f, d, paise / 100, paise % 100);
    }
return 0; }
```

4. Sample Input and Output

Input: none (the programs use fixed values).

Output of the Complete the Code program (**produced by running it with GCC; verify on your machine**):

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

The output of the size and experiment program is the data in tables 2.1 and 2.5 above. On your machine, copy your own output into those tables.

5. Explanation of Output

- `consumption(1200, 1530)`: the difference is 1530 − 1200 = 330 units.
- `consumption(65400, 120)`: in C, `current - previous` is first promoted to `int`, giving 120 − 65400 = **−65280**. The cast `(meter_t)` converts it to `unsigned short`. The C rule for unsigned conversion is "add or subtract 65536 until the value fits in 0..65535". So −65280 + 65536 = **256**.
- **Why 256 is correct:** the meter counted 65400 to 65535 (135 units), then one tick to 0 (1 unit), then 0 to 120 (120 units). Total = 135 + 1 + 120 = **256**. A negative number would be wrong because a meter cannot consume negative units. The wrap-around of the unsigned type does the correction automatically, with no if-else.

6. Alternative Concept and Complexity Analysis: money as double rupees

6.1 What the solution would be

```
double rupees = 50.0;                  /* fixed charge */
rupees += (units - 200) * 4.50;        /* energy charge in double */
/* Rounding must be applied before comparing or printing: */
long long paise = llround(rupees * 100.0);   /* needs <math.h>
*/
```

Rounding would be applied (a) to each bill, converting rupees to whole paise with `llround`, and (b) once more to the grand total before printing. Without rounding, 162.499999999 could print as 162.49 or 162.50 depending on the format.

6.2 Analysis and comparison

Property	long / long long paise	double rupees
Time per addition	O(1)	O(1)
Space per value	4 or 8 bytes	8 bytes
Rounding needed	Never (exact)	Needed every time a result is displayed or compared
Error after n additions	0	Grows with n (see below)

How the error grows (my measurements, adding Rs 162.35 n times):

n	double error (Rs)	float error (Rs)
4,000	5×10^{-8}	5.00
60,000	1.1×10^{-5}	115.00
800,000	1.9×10^{-3}	626,720.00

For double, each addition has a relative error of about 10^{-16}, and the total itself grows with n. So the absolute error grows faster than n (in the worst case about $O(n^2 \cdot \varepsilon)$, where ε is the machine epsilon). From 4,000 to 800,000 bills (200 times more) the error grew about 37,000 times. double happens to stay below one paisa after rounding at these scales, but nothing guarantees it. float fails badly. Integer paise has error 0 at every n.

7. Evidence / Result

- The data-type table (2.2) and the size/range table (2.1).
- Overflow evidence: the 16-bit roll-over result of 256 units.
- Precision evidence: the float error of Rs 5.00 at 4,000 bills and its resolution using integer paise.

TASK 3 - DEFINE THE CONSTANTS

1. Aim

To define every tariff value and limit once, as a named constant, so that a tariff revision is made in a single place only. I also compare `#define`, `const` and `enum`.

2. Explanation and Analysis

2.1 The three ways to define a constant

Method	Example	Best use
#define	#define RATE2_PAISE 225	Simple numeric constants that the preprocessor replaces before compilation
const variable	const long FIXED_CHARGE_PAISE = 5000L;	Typed constants that need type checking and a name in the debugger
enum	enum { SLAB1_UNITS = 100, SLAB2_UNITS = 200 };	A group of related whole-number constants, such as slab limits or categories

2.2 #define constant vs const variable

Point	#define	const variable
Who handles it	Preprocessor (text replacement)	Compiler
Type checking	None. It is only text	Yes. It has a declared type
Scope	From the definition to the end of the file (no block scope)	Normal C scope (block, file)
Debugger	Name not visible, only the replaced number	Name and value visible
Memory	No storage	May occupy storage (often optimised away)
Use as array size in C	Yes	Not in older C. In C11 it makes a variable-length array
Danger	#define X 5+5 gives surprises if not bracketed	None of this kind

2.3 Naming conventions

- Constants are written in UPPER_CASE with underscores, e.g. `FIXED_CHARGE_PAISE`.
- The unit goes into the name (`_PAISE`, `_UNITS`) so that nobody confuses rupees with paise.
- The name describes the meaning, not the value (`RATE2_PAISE`, not `TWO_TWENTY_FIVE`).
- No magic numbers (bare 225 or 450) appear anywhere else in the program.

2.4 Constants used in this assignment

Constant	Value	Meaning
SLAB1_UNITS	100	End of the first slab
SLAB2_UNITS	200	End of the second slab
RATE1_PAISE	0	Slab 1 rate (free)
RATE2_PAISE	225	Rs 2.25 per unit
RATE3_PAISE	450	Rs 4.50 per unit
FIXED_CHARGE_PAISE	5000L	Rs 50 fixed charge

3. C Program / Code

Complete the Code: TODO 1 (macros and main unchanged, only the TODO replaced):

```
#include <stdio.h>

#define SLAB1_UNITS 100        /* first 100 units */
#define SLAB2_UNITS 200        /* units 101 to 200 */
#define RATE1_PAISE 0          /* first slab is free */
#define RATE2_PAISE 225        /* Rs 2.25 per unit */
```

```
#define RATE3_PAISE 450        /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    // TODO 1: Add the energy charge slab by slab using the macros: units
above
    // SLAB2_UNITS at RATE3_PAISE, units from SLAB1_UNITS+1 to SLAB2_UNITS at
    // RATE2_PAISE, and the rest at RATE1_PAISE
    if (units > SLAB2_UNITS) {                      /* units above 200 */
paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;                        /* remaining units are at most 200 */
    }
    if (units > SLAB1_UNITS) {                      /* units 101 to 200 */
paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;                        /* remaining units are at most 100 */
    }
    paise += (long)units * RATE1_PAISE;             /* the first 100 units
*/

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
int n = sizeof tests / sizeof tests[0];
for (int i = 0; i < n; i++) {         long
p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
return 0; }
```

Compile: `gcc -std=c11 -Wall -Wextra -o task3 task3.c && ./task3` (compiled with no warnings when I ran it).

4. Sample Input and Output

Input: none (the test values are inside main).

Output (**produced by running it with GCC; verify on your machine**):

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Tariff revision test: change the single line to `#define RATE2_PAISE 230`, recompile and run:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

5. Explanation of Output

- **0 and 100 units:** both fall in the free slab. Bill = 5000 paise = Rs 50.00, the fixed charge only.
- **150 units:** 50 units are in slab 2. 5000 + 50 × 225 = 5000 + 11,250 = 16,250 paise = Rs 162.50.
- **200 units:** 100 units are in slab 2. 5000 + 100 × 225 = 27,500 paise = Rs 275.00.
- **350 units:** 150 units are above 200 and 100 units are in slab 2. 5000 + 150 × 450 + 100 × 225 = 5000 + 67,500 + 22,500 = 95,000 paise = Rs 950.00.

Units	Slab-2 units	Change	New bill
0, 100	0	none	Rs 50.00
150	50	50 × 5 = 250 paise	Rs 165.00
200	100	100 × 5 = 500 paise	Rs 280.00
350	100	100 × 5 = 500 paise	Rs 955.00

This shows that one edit updated every bill. This is the fix for the "partial tariff revision" fault.

6. Alternative Concept and Complexity Analysis: const or enum instead of #define

6.1 What would change

```
/* Option A: const variables */
const int  SLAB1_UNITS = 100;
const int  SLAB2_UNITS = 200;
const int  RATE2_PAISE = 225;
const int  RATE3_PAISE = 450;
const long FIXED_CHARGE_PAISE =
5000L;

/* Option B: enum (integers only) */ enum { SLAB1_UNITS =
100, SLAB2_UNITS = 200,        RATE1_PAISE = 0, RATE2_PAISE =
225, RATE3_PAISE = 450 };
```

The body of `compute_bill` stays exactly the same. Only the definitions change. The long fixed charge stays as `const long`, because an enum constant is an int.

6.2 Comparison

Point	#define	const	enum
Type checking	None	Yes (declared type)	Integer only
Debugger	Name lost	Name and value visible	Name visible
Memory	None	May use 4 to 8 bytes (often none after optimisation)	None
Can hold a long or double	Yes	Yes	No (int only)
Good for	Quick numeric constants	Typed values	A group of related integer constants
Running time	O(1)	O(1)	O(1)

Is the running time affected? No. A #define is replaced by a literal. A const with a known value is usually folded into the instruction by the compiler, and at worst it costs one memory load. Either way, the bill takes O(1) time. The space difference is at most a few bytes (5 constants × 4 bytes = 20 bytes), which is negligible.

7. Evidence / Result

- One named set of constants (table 2.4) defined in a single place.
- Justified choice: #define for the plain slab and rate numbers (as the Complete the Code requires). const long or an enum are better where type checking, debugger visibility or grouped integers are wanted.

TASK 4 - USE MACROS SAFELY

1. Aim

To write macros for small repeated calculations (rupees to paise, larger of two values, square), to see how the preprocessor expands them, and to understand the pitfalls of unparenthesised macros and of macro arguments that are evaluated twice.

2. Explanation and Analysis

2.1 How a macro is expanded

Before the compiler sees the program, the **preprocessor** performs plain **text replacement**. A function-like macro such as `#define SQUARE(x) ((x) * (x))` is replaced by its body, and each parameter is replaced by the exact text of the argument. The preprocessor does not know about types, values or C syntax. It only copies text. The command `gcc -E file.c` stops after this stage and prints the expanded program, so it is the best way to see what a macro really does.

2.2 The macros I wrote

Macro	Purpose	Safe form
RUPEES_TO_PAISE(r)	Convert rupees to paise	`((long)(r) * PAISE_PER_RUPEE)`
MAX(a, b)	Larger of two values	`(((a) > (b)) ? (a) : (b))`
SQUARE(x)	x multiplied by itself	`((x) * (x))`

The rules for a safe macro: put **every parameter** in parentheses, and put the **whole body** in parentheses.

2.3 Pitfall 1: SQUARE(x) written as x*x

With `#define BAD_SQUARE(x) x * x`, the call `BAD_SQUARE(a + 1)` becomes `a + 1 * a + 1`. Multiplication happens before addition, so for a = 4 the result is 4 + (1 × 4) + 1 = **9**, not 25. The fix is `((x) * (x))`, which expands to `((a + 1) * (a + 1))` = 5 × 5 = **25**. The same problem happens when the whole body is not parenthesised: `10 + BAD_MAX(3, 4)` becomes `10 + 3 > 4 ? 3 : 4`. This is read as `(10 + 3) > 4 ? 3 : 4`, which gives **3** instead of 14.

2.4 Pitfall 2: an argument evaluated twice (side effects)

In `MAX(a, b)` the parameter `a` appears **twice** in the expansion. So `MAX(i++, j)` becomes `(((i++) > (j)) ? (i++) : (j))`, and `i++` is executed twice. With i = 5 and j = 3 the result is 6 and i ends at 7, instead of 5 and 6. Arguments with side effects (`i++`, `--n`, a function call that prints or reads input) must never be passed to a macro. Also, `SQUARE(i++)` would modify i twice in one expression, which is undefined behaviour in C.

2.5 When a function is a better choice than a macro

Situation	Better choice
The argument may have side effects (i++, function calls)	Function (argument evaluated once)
Type checking and warnings are wanted	Function (the compiler checks types)
The body is long or has loops or several statements	Function (macro would copy it everywhere)
Debugging with a debugger or breakpoints	Function (macros are invisible after preprocessing)
The code must be recursive or its address is needed	Function
Very small calculation, same code used for int, long and double	Macro (works for any type)

3. C Program / Code

```
#include <stdio.h>

#define PAISE_PER_RUPEE 100L

/* Safe (fully parenthesised) macros */
#define RUPEES_TO_PAISE(r)  ((long)(r) * PAISE_PER_RUPEE)
#define MAX(a, b)           (((a) > (b)) ? (a) : (b))
#define SQUARE(x)           ((x) * (x))

/* Unsafe versions, kept only to demonstrate the pitfalls */
#define BAD_SQUARE(x)       x * x
#define BAD_MAX(a, b)       a > b ? a : b

/* Alternative: static inline functions */
static inline int max_int(int a, int b)       { return (a > b) ? a : b; }
static inline long square_long(long x)        { return x * x; }
```

```
int main(void)
{     int a =
4;
    printf("1. SQUARE(a + 1)       = %d (correct: 25)\n", SQUARE(a + 1));
printf("2. BAD_SQUARE(a + 1)  = %d (wrong)\n", BAD_SQUARE(a + 1));
printf("3. 10 + BAD_MAX(3, 4) = %d (correct: 14)\n", 10 + BAD_MAX(3, 4));

    int i = 5, j = 3, m;
m = MAX(i++, j);
    printf("4. MAX(i++, j): result = %d, i = %d (expected 5 and 6)\n", m, i);

    i = 5;
    m = max_int(i++, j);
    printf("5. max_int(i++, j): result = %d, i = %d\n", m, i);

    printf("6. RUPEES_TO_PAISE(50) = %ld paise\n",
RUPEES_TO_PAISE(50));     printf("7. square_long(a + 1)  = %ld\n",
square_long(a + 1));     return 0; }
```

Compile and run: `gcc -std=c11 -Wall -Wextra -o task4 task4.c && ./task4` (no warnings).

4. Sample Input and Output

Input: none.

Output (**produced by running it with GCC; verify on your machine**):

```
1. SQUARE(a + 1)       = 25 (correct: 25)
2. BAD_SQUARE(a + 1)  = 9 (wrong)
3. 10 + BAD_MAX(3, 4) = 3 (correct: 14)
4. MAX(i++, j): result = 6, i = 7 (expected 5 and 6)
5. max_int(i++, j): result = 5, i = 6
6. RUPEES_TO_PAISE(50) = 5000 paise7. square_long(a + 1)  = 25
```

gcc -E evidence. I put the macro uses in a short file `mini.c` (a file with `#include <stdio.h>` would print hundreds of lines of system declarations first). The command was `gcc -E mini.c`. The preprocessed result (blank lines removed) is:

```
int main(void)
{
    int a = 4, i = 5, j = 3;
long fixed = ((long)(50) * 100L);
int good = ((a + 1) * (a + 1));
int bad = a + 1 * a + 1;
    int big = (((i++) > (j)) ? (i++) : (j));
int bm = 10 + 3 > 4 ? 3 : 4;
    return 0;
}
```

The `#define` lines and comments have disappeared (they became blank lines), and every macro call has been replaced by its text.

5. Explanation of Output

1. `SQUARE(a + 1)` expands to `((a + 1) * (a + 1))` = 5 × 5 = 25. Correct.
2. `BAD_SQUARE(a + 1)` expands to `a + 1 * a + 1` = 4 + 4 + 1 = 9. Wrong, because `*` binds tighter than `+`.
3. `10 + BAD_MAX(3, 4)` expands to `10 + 3 > 4 ? 3 : 4`. Since 13 > 4 is true, the result is 3, not 14.
4. `MAX(i++, j)`: the condition `i++` compares 5 > 3 (true) and makes i = 6. The true branch then evaluates `i++` again, giving 6 and making i = 7. So the result is 6 and i is 7.
5. `max_int(i++, j)` is a real function. The argument `i++` is evaluated once: a = 5, then i becomes 6. Result 5, i = 6. This is the correct behaviour.
6. `RUPEES_TO_PAISE(50)` = (long)50 × 100 = 5000 paise (Rs 50 fixed charge).
7. `square_long(a + 1)`: the argument is calculated first (5), then squared: 25.

6. Alternative Concept and Complexity Analysis: static inline functions instead of macros

6.1 What the solution is

`MAX` and `SQUARE` are written as `static inline` functions, as in the program above:

```
static inline int  max_int(int a, int b)    { return (a > b) ? a : b; }
static inline long square_long(long x)      { return x * x; }
```

Arguments are evaluated exactly once, types are checked, and the compiler may still insert the body at the call site (inline expansion), so there is no loss of speed. The price is that each function works for one type only (`max_int` for int, `square_long` for long), so separate versions are needed for other types.

6.2 Time and space analysis (k = number of places where the macro or function is used)

Property	Macro	static inline function
Time per use	O(1), no call overhead	O(1). A real call adds a few instructions unless the compiler inlines it
Code size for k uses	O(k): the body is copied at every use	O(1) body plus k call sites (or O(k) if the compiler inlines every call)
Argument evaluation	Text pasted, possibly several times	Exactly once
Type checking	None	Yes

My measurements (compiled to an object file with `gcc -O0 -c`, text size from the `size` command; the test function used the macro or the function k times):

k (uses)	Macro (bytes)	Inline function (bytes)
1	270	333
10	522	621
100	3,312	3,771
1,000	32,112	36,171

The size grows linearly with k for both, about 31.9 bytes per use for the macro and 35.9 bytes per use for the function. For a body as tiny as this one, the call instructions at each call site are about as big as the expanded macro, so the function saves nothing. A function saves space only when its body is large, because the body then exists once while a macro would copy it k times.

Time: I called each version 300 million times. At `-O0`: macro about 0.54 to 0.57 s, inline function about 0.50 to 0.52 s. At `-O2`: macro about 0.09 to 0.10 s, inline function about 0.09 to 0.11 s. The differences are within normal measurement noise, so there is no meaningful speed difference. The macro is not faster, and the function is much safer. Both are O(1) per use.

7. Evidence / Result

- Three fully parenthesised macros (table 2.2) and the matching unsafe versions.
- `gcc -E` evidence of every expansion and an explanation of each pitfall (sections 4 and 5).
- Measured code-size and timing comparison for the alternative concept (section 6.2).

TASK 5 - ORGANISE THE PROGRAM WITH HEADER FILES AND CONDITIONAL COMPILATION

1. Aim

To move the constants and function prototypes into a header file protected by an include guard, to split the program into files, and to add a debugging mode that is switched on only when compiling with `gcc -DDEBUG`.

2. Explanation and Analysis

2.1 Structure of a C program (in the usual order, from top of file to bottom)

Part	Example in my program	Purpose
Comment block	`/* billing.c - billing calculations */`	Says what the file does
Header files (#include)	`#include <stdio.h>`, `#include "billing.h"`	Bring in declarations
Macros / constants (#define)	`SLAB1_UNITS`, `RATE2_PAISE`	Named constants (in billing.h)
Type definitions	`typedef unsigned short meter_t;`	New type names
Global declarations and prototypes	`long compute_bill(unsigned int units);`	Tell the compiler about functions before they are used
main()	main.c	Where execution starts
Other functions	`units_consumed()`, `compute_bill()`	The actual work

My project files

File	Contents
billing.h	Include guard, typedef, limits, tariff constants, prototypes
billing.c	`units_consumed()` and `compute_bill()`, with DEBUG messages
main.c	A small driver program that tests two meter readings

2.2 Include guards

The three lines `#ifndef BILLING_H`, `#define BILLING_H` and `#endif` mean "include this file only if BILLING_H has not been defined yet". The first time the file is read, BILLING_H gets defined. If the header is included again (for example, directly and through another header), the second copy is skipped. Guards are needed because repeated definitions of structures, enums and (in older C) typedefs cause compile errors, and because reading a file twice wastes compile time.

2.3 Conditional compilation with #ifdef DEBUG

The lines between `#ifdef DEBUG` and `#endif` are kept only if the macro DEBUG is defined. The command-line option `-DDEBUG` defines it, so:

- `gcc -std=c11 -Wall -Wextra -o billing main.c billing.c` leaves out all debug lines.
- `gcc -std=c11 -Wall -Wextra -DDEBUG -o billing_dbg main.c billing.c` includes them.

2.4 #include <file> vs #include "file"

Form	Where the preprocessor searches	Use for
`#include <stdio.h>`	The system include directories (and any `-I` directories)	Standard library headers
`#include "billing.h"`	The directory of the current file first, then the system directories	Your own project headers

3. C Program / Code

billing.h

```
/* billing.h - constants and prototypes for the Grama Jyothi billing program */
#ifndef BILLING_H          /* include guard: prevents double inclusion */
#define BILLING_H typedef unsigned short meter_t;      /* 16-

bit meter register */

/* ---- Limits ---- */
#define MAX_READING        65535U    /* largest value the register can show */
```

```
#define MAX_MONTHLY_UNITS  10000U    /* plausibility limit for one household-month
*/
#define MAX_HOUSEHOLD_NO   800000L   /* district size */
/* ---- Tariff (all money in paise) ---- */
#define SLAB1_UNITS        100U
#define SLAB2_UNITS        200U
#define RATE1_PAISE        0L
#define RATE2_PAISE        225L
#define RATE3_PAISE        450L
#define FIXED_CHARGE_PAISE 5000L

/* ---- Function prototypes ---- */
unsigned int units_consumed(meter_t previous, meter_t
current); long compute_bill(unsigned int units); #endif /*
BILLING_H */
```

billing.c

```
/* billing.c - billing calculations */
#include <stdio.h>
#include "billing.h"

/* Units used, correct even after the 16-bit register rolls over (65535 -> 0) */
unsigned int units_consumed(meter_t previous, meter_t current)
{
    unsigned int units = (meter_t)(current - previous);
#ifdef DEBUG
    printf("[DEBUG] previous=%u current=%u units=%u\n",
(unsigned)previous, (unsigned)current, units);
#endif
return units; }

/* Bill in paise, slab by slab */
long compute_bill(unsigned int
units)
{
    long paise = FIXED_CHARGE_PAISE;
unsigned int remaining = units;

    if (remaining > SLAB2_UNITS) {
        paise += (long)(remaining - SLAB2_UNITS) * RATE3_PAISE;
#ifdef DEBUG
        printf("[DEBUG] slab3 units=%u charge so far=%ld paise\n",
remaining - SLAB2_UNITS, paise);
#endif
        remaining = SLAB2_UNITS;
    }
    if (remaining > SLAB1_UNITS) {
        paise += (long)(remaining - SLAB1_UNITS) * RATE2_PAISE;
#ifdef DEBUG
        printf("[DEBUG] slab2 units=%u charge so far=%ld paise\n",
remaining - SLAB1_UNITS, paise);
#endif
        remaining = SLAB1_UNITS;
    }
    paise += (long)remaining * RATE1_PAISE;
#ifdef DEBUG
    printf("[DEBUG] total=%ld paise\n", paise);
#endif
return paise; }
```

main.c

```
/* main.c - small driver for Task 5 */
#include <stdio.h>
#include "billing.h"

int main(void)
{
    meter_t prev[] = {1200, 65400};     meter_t
curr[] = {1530, 120};     int n = (int)(sizeof
prev / sizeof prev[0]);

    for (int i = 0; i < n; i++) {
        unsigned int units = units_consumed(prev[i], curr[i]);
long bill = compute_bill(units);
        printf("Units = %u, bill = Rs %ld.%02ld\n", units, bill / 100, bill % 100);
    }
return 0;
```

```
}
```

Compile commands:

- Normal: `gcc -std=c11 -Wall -Wextra -o billing main.c billing.c`
- Debug: `gcc -std=c11 -Wall -Wextra -DDEBUG -o billing_dbg main.c billing.c` Both

compiled with no warnings.

4. Sample Input and Output

Input: none (the readings are inside main).

Normal build, `./billing` (**produced by running it; verify on your machine**):

```
Units = 330, bill = Rs 860.00
Units = 256, bill = Rs 527.00
```

Debug build, `./billing_dbg`:

```
[DEBUG] previous=1200 current=1530 units=330
[DEBUG] slab3 units=130 charge so far=63500 paise
[DEBUG] slab2 units=100 charge so far=86000 paise
[DEBUG] total=86000 paise
Units = 330, bill = Rs 860.00
[DEBUG] previous=65400 current=120 units=256
[DEBUG] slab3 units=56 charge so far=30200 paise
[DEBUG] slab2 units=100 charge so far=52700 paise
[DEBUG] total=52700 paise
Units = 256, bill = Rs 527.00
```

5. Explanation of Output

- **Reading 1200 to 1530:** units = 330. Units above 200 = 130, charged at 450 paise: 5000 + 130 × 450 = 5000 + 58,500 = 63,500 paise (the debug line "charge so far=63500"). The next 100 units (101 to 200) cost 100 × 225 = 22,500, giving 86,000 paise. The first 100 units are free. Bill = Rs 860.00.
- **Reading 65400 to 120 (roll-over):** units = 256. Above 200 = 56 units: 5000 + 56 × 450 = 30,200 paise. Plus 100 × 225 = 22,500 gives 52,700 paise = Rs 527.00.
- The `[DEBUG]` lines appear only in the debug build, which shows that the code between `#ifdef DEBUG` and `#endif` was included or removed by the preprocessor. The final bills are identical in both builds.

6. Alternative Concept and Complexity Analysis: a run-time debug variable

6.1 What the solution is

Instead of `#ifdef DEBUG`, a global variable decides whether to print: `int debug_on = 0;`

```
/* run-time switch, set to 1 by e.g. a -d option */

    if (debug_on) {
        printf("[DEBUG] total=%ld paise\n",
paise);     }
```

The debug code is always compiled. It can be switched on or off while the program is running, without recompiling. I tested this version by replacing each `#ifdef DEBUG ... #endif` pair with `if (debug_on) { ... }`. With `debug_on = 0` it printed the same two normal lines as the normal build.

6.2 Time and space analysis

Property	#ifdef DEBUG	Run-time `if (debug_on)`
Extra time per call	None when DEBUG is off (code removed)	One load, one compare and one branch for each test, O(1)
Time when debugging is on	Printing dominates in both	Printing dominates in both
Debug code in executable	Only if compiled with -DDEBUG	Always present
Change debugging without recompiling	No	Yes

My measurements (billing functions called 100 million times with debugging off, two runs each):

Build	-O0 time	-O2 time
#ifdef (DEBUG off)	0.230 to 0.236 s	0.105 s
Run-time flag (off)	0.255 to 0.276 s	0.161 to 0.202 s

Build	Executable text size
Normal build (no DEBUG)	2,024 bytes
Compiled with -DDEBUG	2,341 bytes

Build	Executable text size
Run-time flag (debugging off)	2,381 bytes

The test costs a small constant time, which is still O(1) per call, but in this very short function it is measurable: about 8 to 20 percent slower at `-O0` and about 50 to 90 percent slower at `-O2` (the timings were noisy, so treat them as approximate). The `#ifdef` build with debugging off contains none of the debug code (2,024 bytes). The run-time version always carries it (2,381 bytes) plus the variable itself (4 bytes). So `#ifdef` is better for the final release (smaller and faster), and the run-time flag is more convenient when debugging must be turned on without recompiling.

7. Evidence / Result

- A header file with include guards (billing.h), a working DEBUG mode (outputs above) and a table of the program structure.
- Screenshots to add: both compile commands, and both outputs.
- Measured size and time comparison for the alternative concept.

TASK 6 - READ AND PRINT DATA CORRECTLY

1. Aim

To design the program's input and output with `scanf` and `printf`: use the correct format specifier for every type, show what happens with a wrong specifier, validate input using the return value of `scanf`, and print an amount stored in paise as rupees and paise (for example Rs 162.50).

2. Explanation and Analysis

2.1 Format specifiers

Type	printf	scanf	Used for in this project
int	`%d`	`%d`	Test values
unsigned int	`%u`	`%u`	Units consumed, MAX_READING
unsigned short	`%hu` (or `%u` after casting)	`%hu`	meter_t (I read into long first, see 2.3)
long	`%ld`	`%ld`	Household number, readings, bills in paise
long long	`%lld`	`%lld`	District totals in paise
float	`%f`	`%f`	Only in the float experiments
double	`%f` (or `%lf`)	`%lf` (not %f)	Only in the double experiments
char	`%c`	`%c`	Category letter
string	`%s`	`%s`	Text such as "01530"
size_t (result of sizeof)	`%zu`	-	Printing sizes

Note that for `double`, `printf` accepts `%f`, but `scanf` must use `%lf`.

2.2 What happens with a wrong specifier (my test)

The test program used `long big = 5000000000L;` and `long v = -1L;`. GCC gave these warnings with `-Wall -Wextra`:

```
spec.c:6:39: warning: format '%d' expects argument of type 'int',
but argument 2 has type 'long int' [-Wformat=]
spec.c:11:13: warning: format '%d' expects argument of type 'int *',
but argument 2 has type 'long int *' [-Wformat=]
```

The program still compiled and ran. Its output was:

```
Wrong  (%d  with long): 705032704
Right  (%ld with long): 5000000000
Type 5 and press Enter: v after scanf("%d") = -4294967291
```

Test	Result	Reason
`printf("%d", big)` with big = 5,000,000,000	**705032704**	`%d` reads only the low 32 bits: 5,000,000,000 − 4,294,967,296 = 705,032,704
`scanf("%d", &v)` into a long, typing 5	v = **-4294967291**	`%d` writes only 4 bytes. The upper 4 bytes of v stay all 1s (from -1), so v = -2^{32} + 5 = −4,294,967,291

Both errors are silent. This is why the warnings must never be ignored. The results shown are for a 64-bit Linux system. On other systems the printed value of a wrong specifier can be different, because the behaviour is undefined.

2.3 Checking the return value of scanf `scanf` returns the number of items it successfully read, or `EOF` when the input has ended.

- If the user types `abc` for `%ld`, scanf returns **0** and leaves `abc` in the input. If the program does not remove it, the next `scanf` fails again and the program loops forever. So on failure I discard the rest of the line with a small `discard_line()` function and ask again.
- A reading is first read into a **long** (`%ld`) and then range-checked (0 to 65535). If it were read directly with `%hu`, a typed `-5` or `70000` could not be detected reliably, because `scanf` converts or wraps such values silently (the result is implementation-dependent). Reading into the wider type and checking the range detects both.
- The category is read with `" %c"`. The space before `%c` skips blanks and newlines left by earlier input.

2.4 Printing rupees and paise from an integer amount in paise For an amount `p` in paise

(never negative for a bill):

- Rupees = `p / 100` (integer division).
- Paise = `p % 100` (remainder).
- Print with `printf("Rs %ld.%02ld", p / 100, p % 100);`

The `02` is important. For p = 16205 the correct output is Rs 162.05, but `%ld.%ld` would print `162.5` (the leading zero is lost). For p = 16250, `%02ld` gives `162.50`. No floating-point number is used, so no rounding error can occur.

3. C Program / Code

The program uses the same `billing.h` and `billing.c` as Task 5.

```
/* task6.c - reads household data with scanf, validates it and prints bills.
Compile: gcc -std=c11 -Wall -Wextra -o task6 task6.c billing.c            */
#include <stdio.h>
#include "billing.h"

/* Throw away the rest of the current input line. */
static void discard_line(void)
{     int
ch;
    while ((ch = getchar()) != '\n' && ch != EOF) {
        /* skip */
    }
}

/* Reads one whole number into *value.
   Returns 1 on success, 0 when the input has ended (EOF).
   If the user types letters, scanf returns 0, so we complain and ask again. */
static int read_number(const char *prompt, long *value)
{     for
(;;) {
        printf("%s", prompt);
        int count = scanf("%ld", value);   /* %ld because the variable is long
*/         if (count == EOF) {             return 0;
        }
        if (count == 1) {
discard_line();
return 1;
        }
        printf("  Invalid input: please type digits only.\n");
discard_line();
    }
}

int main(void)
{
    long household, previous, current;
char category;

    while (read_number("Household number: ", &household)) {
if (household < 1 || household > MAX_HOUSEHOLD_NO) {
            printf("  Error: household number must be 1 to %ld.\n\n", MAX_HOUSEHOLD_NO);
continue;
        }
        printf("Category (D or C): ");
        if (scanf(" %c", &category) != 1) {          /* the space skips blanks */
break;
        }
        discard_line();
        if (category != 'D' && category != 'C') {
printf("  Error: category must be D or C.\n\n");
continue;
        }
        if (!read_number("Previous reading: ", &previous) ||
!read_number("Current reading: ", &current)) {
break;
        }
        /* Read into long first, so that -5 or 70000 can be detected.
*/         if (previous < 0 || previous > (long)MAX_READING ||
current < 0 || current > (long)MAX_READING) {
            printf("  Error: a reading must be between 0 and %u.\n\n",
MAX_READING);             continue;
```

```
        }
        unsigned int units = units_consumed((meter_t)previous, (meter_t)current);
if (units > MAX_MONTHLY_UNITS) {
            printf("  Error: current < previous and not a valid roll-over.\n\n");
            continue;
        }
        long total  = compute_bill(units);
long energy = total - FIXED_CHARGE_PAISE;
        printf("\n  Household %ld (%c)\n", household, category);
printf("  Units consumed : %u\n", units);
        printf("  Fixed charge   : Rs %ld.%02ld\n", FIXED_CHARGE_PAISE / 100, FIXED_CHARGE_PAISE %
100);         printf("  Energy charge  : Rs %ld.%02ld\n", energy / 100, energy % 100);
printf("  Total bill     : Rs %ld.%02ld\n\n", total / 100, total % 100);
    }
return 0; }
```

Compile: `gcc -std=c11 -Wall -Wextra -o task6 task6.c billing.c` (no warnings).

4. Sample Input and Output

Input file `in1.txt` (run with `./task6 < in1.txt`). It contains letters, a negative reading, an invalid roll-over and an out-of-range reading:

```
abc
1042
D
1200
1530
2077
C
65400
120
3001
D
-5
100
3002
D
500
200
3003
D
70000
10
```

Output (**produced by running it; verify on your machine**). When the input comes from a file, the prompts run together on one line because no Enter key is echoed. When you type interactively, each prompt appears on its own line. (In this PDF a few long output lines are wrapped to fit the page.)

```
Household number:   Invalid input: please type digits only.
Household number: Category (D or C): Previous reading: Current reading:
  Household 1042 (D)
  Units consumed : 330
  Fixed charge   : Rs 50.00
  Energy charge  : Rs 810.00
  Total bill     : Rs 860.00

Household number: Category (D or C): Previous reading: Current reading:
  Household 2077 (C)
  Units consumed : 256
  Fixed charge   : Rs 50.00
  Energy charge  : Rs 477.00
  Total bill     : Rs 527.00

Household number: Category (D or C): Previous reading: Current reading:
    Error: a reading must be between 0 and 65535.

Household number: Category (D or C): Previous reading: Current reading:
    Error: current < previous and not a valid roll-over.

Household number: Category (D or C): Previous reading: Current reading:
    Error: a reading must be between 0 and 65535.

Household number:
```

5. Explanation of Output

Input	What happened	Result
`abc` as household number	`scanf("%ld")` returned 0, so the program printed the error, discarded the line and asked again	Re-prompt, then 1042 accepted

Input	What happened	Result
1042, D, 1200, 1530	Units = 330. 5000 + 130 × 450 + 100 × 225 = 86,000 paise. Energy = 86,000 − 5,000 = 81,000 paise	Total Rs 860.00 (energy Rs 810.00)
2077, C, 65400, 120	Roll-over, units = 256. 5000 + 56 × 450 + 100 × 225 = 52,700 paise	Total Rs 527.00 (energy Rs 477.00)
3001, previous = −5	Range check 0 to 65535 failed	Error message (negative reading rejected)
3002, 500 then 200	(meter_t)(200 − 500) = 65,236, which is above MAX_MONTHLY_UNITS (10,000)	Error: not a valid roll-over
3003, previous = 70000	Above 65535, so rejected	Error message
End of input	`scanf` returned EOF, so the loop ended	Program stops normally

Each money value is printed with `%ld.%02ld` from integer paise.

6. Alternative Concept and Complexity Analysis: fgets() with strtol()

6.1 What the solution is

Each input line is first read as a whole into a character buffer with `fgets`, and then converted with `strtol`, which tells exactly where the conversion stopped:

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

/* fgets + strtol: returns 1 on success, 0 on invalid text, -1 on EOF */
static int read_number_line(const char *prompt, long *value)
{
    char line[64];
char *end;

    printf("%s", prompt);
    if (fgets(line, sizeof line, stdin) == NULL) {
return -1;
    }
    line[strcspn(line, "\n")] = '\0';      /* remove the newline */
errno = 0;
    long n = strtol(line, &end, 10);
    if (end == line || *end != '\0' || errno == ERANGE) {
        return 0;                          /* no digits, trailing junk or too big
*/
    }
    *value =
n;     return
1; }
```

I tested it (compiled with no warnings) with the six input lines `1530`, `12abc`, `abc`, an empty line, `99999999999999999999` and `65535`. The results were: accepted 1530, rejected, rejected, rejected, rejected, accepted 65535. So letters, trailing junk (`12abc`), an empty line and a number that is too large are all detected. With my `scanf` version, `12abc` would be accepted as 12 because `scanf` stops reading at the first letter and the rest of the line is simply discarded. **6.2 Time, space and error recovery**

Property	scanf("%ld")	fgets + strtol
Time for a line of length L	O(L): reads characters until a non-digit, then discard_line() skips the rest	O(L): fgets copies L characters, strtol scans them
Extra buffer space	None in my code (only the stdio buffer, which both approaches use)	A line buffer, 64 bytes in my code, so O(B)
Detects trailing junk (12abc)	No (needs extra code)	Yes (`*end != '\0'`)

Detects number too large	Not reliably	Yes (`errno == ERANGE`)
Recovery after invalid input	Bad characters stay in the input, so the program must discard them or it loops forever	The whole line is already consumed, so the input is always in a clean state
Handles lines longer than the buffer	Not an issue	Needs extra care (the rest of the line stays in the input)

The speed of the two approaches is the same, O(L), and L is only a few characters, so both are practically instant. `fgets` plus `strtol` costs a little more memory, but it is more robust, which matters when readings are typed by hand for 800,000 households.

7. Evidence / Result

- The format specifier table (2.1) and the wrong-specifier test with its two compiler warnings and results (2.2).
- Input validation using the return value of `scanf`: the `abc` case and the range checks (sections 4 and 5).
- Correctly formatted bills: Rs 860.00 and Rs 527.00, printed from integer paise with `%ld.%02ld`.

TASK 7 - CONTROL TYPE CONVERSION AND PRECISION

1. Aim

To find every place in the program where values of different types are combined, to understand implicit conversion, integer division, explicit casting and rounding, and to demonstrate at least three expressions that give an unexpected result.

2. Explanation and Analysis

2.1 Conversion rules in C

Rule	Meaning	Example in my program
Integer promotion	Any type narrower than `int` (char, short, unsigned short) is converted to `int` before arithmetic	`current - previous` with two `meter_t` values is calculated in `int`
Usual arithmetic conversions	When two operands have different types, both are converted to a common type. A signed value mixed with an unsigned value of the same size becomes **unsigned**	`int` compared with `unsigned int`
Assignment conversion	The right side is converted to the type of the variable on the left	`unsigned int units = (meter_t)(...)`
Explicit cast	The programmer asks for a conversion with `(type)`	`(long)(remaining - SLAB2_UNITS)`
Integer division	If both operands are integers, the result is an integer and the fraction is thrown away	`p / 100`

2.2 Every conversion point in my billing program

#	Where	Types involved	Safe?	Why
1	`current - previous`	two `unsigned short` promoted to `int`, result `int` (can be negative)	Yes, by design	The negative value is fixed by the next cast
2	`(meter_t)(current previous)`	`int` to `unsigned short`	Yes	C defines the result as the value modulo 65536, which is the roll-over rule
3	`unsigned int units = ...`	`unsigned short` to `unsigned int`	Yes	Widening, no loss
4	`remaining > SLAB2_UNITS`	`unsigned int` and `200U`	Yes	Both unsigned, so no sign surprise. This is why the constants have a `U` suffix
5	`(long)(remaining - SLAB2_UNITS) * RATE3_PAISE`	`unsigned int` cast to `long`, times `long`	Yes	At most 10,000 units, so it fits even in a 32-bit `long`
6	`previous > (long)MAX_READING`	`long` and `unsigned int` cast to `long`	Yes	The explicit cast stops a negative `long` from being turned into a huge unsigned number on systems where `long` is 32 bits
#	Where	Types involved	Safe?	Why
7	`(meter_t)previous`	`long` to `unsigned short`	Yes, only after the range check	Values are already known to be 0 to 65535
8	`total FIXED_CHARGE_PAISE`	`long` and `long`	Yes	Same type
9	`grand_total += total`	`long` added to `long long`	Yes	Widening. The 64-bit total cannot overflow at district scale

10	`bill / 100` and `bill % 100`	`long`	Yes	Intended integer division, used with `%02ld`
11	`printf("%u", units)` and `%ld`, `%lld`	each value matches its specifier	Yes	See Task 6

2.3 Integer division and where it gives a wrong result

`p / 100` with p = 16250 gives 162, and `p % 100` gives 50, so the pair `162` and `50` is exact. The mistake is to use the division alone. For example `double rupees = p / 100;` first divides two integers (162) and only then converts to `double`, so the result is **162.00** instead of 162.50. The fix is either to keep integers and print with `%ld.%02ld`, or to write `p / 100.0`. Another place is an average such as `total / count`. It drops the fraction and also crashes when `count` is 0 (see Task 8).

2.4 Explicit casting and casting a large long to int

A cast tells the compiler "I know what I am doing". It does not check the value. When a `long` that does not fit in a 32-bit `int` is cast, GCC keeps only the low 32 bits. For example `(int)5000000000L` gives **705,032,704**. The largest `int` is 2,147,483,647 paise, which is only Rs 21,474,836.47, and a district total is about 13,000,000,000 paise (Task 1). So a total must **never** be cast to `int`. In my program, only small values that have already been range-checked are cast down (for example `(meter_t)previous`).

2.5 Rounding rule

The tariff has whole-paise rates (225 and 450 paise per unit), so the basic bill never produces a fractional paisa. Fractions appear only when a percentage is applied (such as the 20% peak charge in Task 12) or when watt-hours are converted to units. My rounding rule for the whole assignment is:

1. Keep every amount in integer paise. Never use `float` or `double` for money.
2. Multiply first and divide last, so that no digits are lost: `charge * 120 / 100`, not `charge * 1.2`.
3. Round **half up** to the nearest paisa, **once** for each charge, with the integer formula `(numerator + denominator / 2) / denominator`.
4. Never round a running total.

```
/* Round half up, in integer arithmetic (for non-negative values) */
static long round_div(long numerator, long denominator)
{
    return (numerator + denominator / 2) /
denominator; }
```

2.6 Three or more expressions with unexpected results (all measured)

#	Expression	Beginner expects	Actual result	Reason
1	`double r = p / 100;` with p = 16250	162.50	**162.00**	Integer division happens first, the fraction is lost before the conversion to double
2	`current - previous` (120 and 65400, both `meter_t`)	256	**-65280**	Both are promoted to `int`, so the subtraction is done in `int` and gives a negative number. The cast `(meter_t)` turns it into 256
3	`-1 < 1u`	1 (true)	**0 (false)**	The usual arithmetic conversions turn -1 into the unsigned value 4,294,967,295. GCC warns with `-Wsign-compare`
4	`(int)5000000000L`	5000000000	**705032704**	The value does not fit in 32 bits, so only the low 32 bits are kept
5	`(long)(4.35 * 100)`	435	**434**	4.35 is stored in binary as 4.3499999..., and the cast truncates. Adding 0.5 before the cast gives 435
#	Expression	Beginner expects	Actual result	Reason
6	`short c = 30000 + 30000;`	60000	**-5536**	The sum is 60000 in `int`, but a `short` holds only -32768 to 32767, so it wraps: 60000 - 65536 = -5536

7	`float f = 16777217;`	16777217	**16777216**	A `float` has a 24-bit mantissa and cannot hold every integer above 2^{24} = 16,777,216
8	20% extra on 12343 paise, `12343 * 120 / 100`	14812 (14811.6 rounded)	**14811** (truncated)	Integer division truncates. `round_div(12343 * 120, 100)` gives 14812

A separate small test also showed that `(long)(2.30 * 100)` gives **229**, not 230. This matters for the tariff revision test of Task 3: if a rate of Rs 2.30 were calculated through `double`, every bill would be wrong by a paisa per unit, which is why `RATE2_PAISE` is a whole number of paise.

3. C Program / Code

```
#include <stdio.h> typedef

unsigned short meter_t;

/* Rounding rule: round half up, once, in integer arithmetic (non-negative values) */
static long round_div(long numerator, long denominator)
{
    return (numerator + denominator / 2) /
denominator; }

int main(void)
{
    /* 1. Integer division */
    long p = 16250;                     /* Rs 162.50 in paise */
    double wrong_rupees = p / 100;      /* int division first, then converted
*/     double right_rupees = p / 100.0;    /* floating division */
printf("1. p / 100 = %ld, stored in double = %.2f, p / 100.0 = %.2f\n",
p / 100, wrong_rupees, right_rupees);

    /* 2. Integer promotion of unsigned short */
meter_t previous = 65400, current = 120;
    printf("2. current - previous = %d (promoted to int), as meter_t = %u\n",
current - previous, (unsigned)(meter_t)(current - previous));

    /* 3. Signed versus unsigned comparison
*/     int s = -1;     unsigned int u = 1;
    printf("3. (-1 < 1u) is %d (expected 1)\n", s < u);

    /* 4. Casting a large long to int */
long big = 5000000000L;
    printf("4. (int)5000000000L = %d\n", (int)big);

    /* 5. Fractional rupees truncated instead of rounded */
double rupees = 4.35;
    printf("5. (long)(4.35 * 100) = %ld, with +0.5 = %ld, exact paise = 435\n",
(long)(rupees * 100.0), (long)(rupees * 100.0 + 0.5));

    /* 6. short overflow on assignment
*/     short a = 30000, b = 30000;
short c = (short)(a + b);     int d = a
+ b;
    printf("6. 30000 + 30000 as short = %d, as int = %d\n", c, d);

    /* 7. float cannot hold every integer above 2^24 */
float f = 16777217.0f;
    printf("7. float 16777217 prints as %.0f\n", f);

    /* 8. Rounding rule demo: 20%% extra on 12343 paise = 14811.6 */
long charge = 12343;
    printf("8. truncated = %ld, rounded half up = %ld\n",
charge * 120 / 100, round_div(charge * 120, 100));
return 0; }
```

Compile: `gcc -std=c11 -Wall -Wextra -o task7 task7.c`

GCC gave exactly one warning, for expression 3, which is deliberate:

```
task7.c:28:51: warning: comparison of integer expressions of different signedness:
   'int' and 'unsigned int' [-Wsign-compare]
```

How this warning is resolved in the real program: the billing code never mixes signed and unsigned values. Units are `unsigned int`, the constants have a `U` suffix (`200U`), and when a `long` is compared with an unsigned constant, the constant is cast first: `(long)MAX_READING`. All billing programs of Parts 1 to 3 compile with no warnings.

4. Sample Input and Output

Input: none.

Output (**produced by running it with GCC; verify on your machine**):

```
1. p / 100 = 162, stored in double = 162.00, p / 100.0 = 162.50
2. current - previous = -65280 (promoted to int), as meter_t = 256
3. (-1 < 1u) is 0 (expected 1)
4. (int)5000000000L = 705032704
5. (long)(4.35 * 100) = 434, with +0.5 = 435, exact paise = 435
6. 30000 + 30000 as short = -5536, as int = 60000
7. float 16777217 prints as 16777216
8. truncated = 14811, rounded half up = 14812
```

5. Explanation of Output

- **Line 1:** `p / 100` is 162. Stored in a `double`, it prints 162.00. Dividing by `100.0` gives 162.50.
- **Line 2:** `current - previous` is -65280 because of integer promotion. The cast `(meter_t)` wraps it to -65280 + 65536 = 256.
- **Line 3:** the `int` -1 is converted to `unsigned int` before the comparison, so the test is "4,294,967,295 < 1", which is false.
- **Line 4:** 5,000,000,000 - 4,294,967,296 = 705,032,704.
- **Line 5:** truncation gives 434, adding 0.5 first gives 435.
- **Line 6:** the same sum is 60,000 as an `int` but -5,536 as a `short`.
- **Line 7:** 16,777,217 needs 25 bits, but a `float` keeps 24, so the odd number is rounded to 16,777,216.
- **Line 8:** 12343 × 120 = 1,481,160. Dividing by 100 gives 14,811.6. Truncation keeps 14,811. The half-up rule gives (1,481,160 + 50) / 100 = 14,812.

6. Alternative Concept and Complexity Analysis: all calculations in float, rounded once at the end

6.1 What the solution is

Every bill is calculated in `float` rupees, the bills are added into a `float` total, and the result is converted to paise only once, at the very end:

```
static float bill_float(unsigned int units, float rate2, float rate3)
{
    float b = 50.0f;                                    /* fixed charge in rupees
*/     if (units > 200U) { b += (float)(units - 200U) * rate3; units = 200U; }
if (units > 100U) { b += (float)(units - 100U) * rate2; }     return b;
}
/* total_f += bill_float(units, 2.25f, 4.50f);  for every bill, then at the end: */
long long paise = (long long)((double)total_f * 100.0 + 0.5);   /* rounded once */
```

6.2 My measurement. For n bills, the units of bill i were `(i * 37) % 400`, which spreads the bills over all three slabs. I compared three totals: exact integer paise, `float` and `double`. The test was run with the real rate (Rs 2.25) and with the revised rate of Task 3 (Rs 2.30).

Rate 2	n (bills)	Exact total (integer paise)	float, rounded once: difference	double, rounded once: difference
Rs 2.25	4,000	Rs 1,656,875.00	**0 paise**	0 paise
Rs 2.25	60,000	Rs 24,853,125.00	**-266,700 paise (Rs 2,667.00 short)**	0 paise
Rs 2.25	800,000	Rs 331,375,000.00	**+29,156,000 paise (Rs 291,560.00 too much)**	0 paise
Rs 2.30	4,000	Rs 1,669,350.00	**-12 paise**	0 paise
Rs 2.30	60,000	Rs 25,040,250.00	**-270,000 paise (Rs 2,700.00 short)**	0 paise
Rs 2.30	800,000	Rs 333,870,000.00	**+25,220,800 paise (Rs 252,208.00 too much)**	0 paise

What the numbers show

- With Rs 2.25 and Rs 4.50, `float` is exactly right at 4,000 bills. This is a lucky accident: 2.25 and 4.50 are multiples of 0.25, which binary can store exactly, and the total (about 1.66 million rupees) is below 2^{22} = 4,194,304, where every quarter-rupee can still be stored.
- As soon as the total passes about 16.7 million, the spacing between neighbouring `float` values becomes 2 or more, so every added bill is rounded, and the error becomes thousands of rupees at taluk scale and hundreds of thousands of rupees at district scale.
- With the revised rate of Rs 2.30, `float` is already wrong at village scale (-12 paise), because 2.30 has no exact binary form.
- `double` happened to give the exact answer after the single final rounding, because its error (about 10^{-3} rupee at 800,000 bills, see Task 2) stays below half a paisa. This is not guaranteed for larger sums or other tariffs, whereas integer paise is exact by construction.
- The error of integer paise is **0 for every n**. The error of `float` does not grow smoothly. It is zero while the total is small and then jumps once the total crosses the limit of the 24-bit mantissa.

6.3 Time and space per value

Property	Integer paise	float rupees	double rupees
Time per bill	O(1)	O(1)	O(1)
Time for 100 million bills (-O0)	0.275 s	0.358 s	0.295 s
Time for 100 million bills (-O2)	0.152 s	0.094 s	0.093 s
Space per value	8 bytes (`long long` here)	4 bytes	8 bytes
Error after n bills	0	grows, then jumps (see above)	below one paisa here, no guarantee

The speed is the same order for all three, and all are O(1) per value, so speed is not a reason to choose floating point. `float` saves 4 bytes per value, which for 800,000 totals is only 3.2 MB, a tiny saving compared with the wrong answers it can produce.

7. Evidence / Result

- Identified conversion points (table 2.2).
- Eight demonstrated surprises with explanations (tables 2.6 and section 5), more than the three required.
- A consistent rounding rule (section 2.5) and the float-versus-paise measurements (section 6.2).

TASK 8 - TRACE THE COMPILATION PROCESS

1. Aim

To follow the Task 5 program (`main.c`, `billing.c`, `billing.h`) through the stages of compilation, to classify compile-time, link-time and run-time errors with real examples, and to explain the warnings given by `gcc -Wall -Wextra`.

2. Explanation and Analysis

2.1 The four stages

Stage	Command	Output file	What it does
1. Preprocessing	`gcc -E main.c -o main.i`	main.i (text)	Handles every line starting with `#`: inserts header files, replaces macros, removes comments, keeps or drops `#ifdef` blocks
2. Compilation	`gcc -std=c11 -S billing.c -o billing.s`	billing.s (assembly text)	Checks the C syntax and translates it to assembly language
3. Assembly	`gcc -std=c11 -c billing.c -o billing.o`	billing.o (machine code)	Converts assembly to an object file. Calls to functions in other files are left unresolved
4. Linking	`gcc main.o billing.o -o billing`	billing (executable)	Joins the object files and libraries and fills in the addresses of all functions

2.2 Stage 1: preprocessed output (gcc -E)

Line counts: `main.c` has 17 lines and becomes **839 lines** in `main.i`. `billing.c` has 43 lines and `billing.h` has 24, and `billing.c` becomes **865 lines** in `billing.i`. Most of the extra lines are the contents of `stdio.h`. The start of `main.i` shows the line markers that the preprocessor adds:

```
# 0 "main.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "main.c"

# 1 "/usr/include/stdio.h" 1 3 4
```

The end of `main.i` shows that `main()` is unchanged, but `billing.h` has been pasted in above it (so the prototypes are visible) and the two comments are gone:

```
long compute_bill(unsigned int units);
# 4 "main.c" 2

int main(void)
{
    meter_t prev[] = {1200, 65400};
meter_t curr[] = {1530, 120};
    int n = (int)(sizeof prev / sizeof prev[0]);

    for (int i = 0; i < n; i++) {
        unsigned int units = units_consumed(prev[i], curr[i]);
long bill = compute_bill(units);
        printf("Units = %u, bill = Rs %ld.%02ld\n", units, bill / 100, bill % 100);
    }
return 0; }
```

In `billing.i` the tariff macros have been replaced by their values. The command `grep -n "paise +=\|long paise\|remaining >" billing.i` printed:

```
841:    long paise = 5000L;
844:    if (remaining > 200U) {
845:        paise += (long)(remaining - 200U) * 450L;
852:    if (remaining > 100U) {
853:        paise += (long)(remaining - 100U) * 225L;
860:    paise += (long)remaining * 0L;
```

What the preprocessor removed or replaced:

- Every `#include` line was replaced by the full text of that file.

- Every `#define` constant (`FIXED_CHARGE_PAISE`, `SLAB2_UNITS`, `RATE3_PAISE` and so on) was replaced by its value, as shown above, and the `#define` lines themselves disappeared.
- All comments were removed.
- The `#ifndef BILLING_H` guard was evaluated, and the `#ifdef DEBUG` blocks were deleted completely (in a normal build).

2.3 Stage 2: assembly (gcc -S) `billing.s` has 86 lines. The start of the

function `compute_bill` is:

```
compute_bill:
.LFB1:

.cfi_startproc
endbr64         pushq
%rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
movq    %rsp, %rbp
        .cfi_def_cfa_register 6
movl    %edi, -20(%rbp)
movq    $5000, -8(%rbp)
movl    -20(%rbp), %eax
movl    %eax, -12(%rbp)
cmpl    $200, -12(%rbp)
jbe     .L4         movl    -
12(%rbp), %eax          subl
$200, %eax          movl
%eax, %eax          imulq
$450, %rax, %rax          addq
%rax, -8(%rbp)          movl
$200, -12(%rbp)
.L4:          cmpl    $100, -
12(%rbp)
```

This shows that the constants are now plain numbers inside the instructions: `movq $5000` stores the fixed charge, `cmpl $200` is the slab limit test, `imulq $450` multiplies by the rate. The instruction `jbe` ("jump if below or equal") is the unsigned comparison that comes from `remaining > 200U`.

2.4 Stage 3: object files (gcc -c)

File	Size	Symbols from `nm`
billing.o	1,456 bytes	`T compute_bill`, `T units_consumed` (T = defined here)
main.o	1,992 bytes	`T main`, and `U` (undefined) `compute_bill`, `units_consumed`, `printf`, `__stack_chk_fail`

The `U` entries in `main.o` are exactly the functions that the linker must still find. `file main.o` reports `ELF 64-bit LSB relocatable`. **2.5 Stage 4: linking**

`gcc main.o billing.o -o billing` produced an executable of 16,120 bytes (`ELF 64-bit LSB pie executable, dynamically linked`). The linker matched every `U` symbol of `main.o` to the `T` symbols of `billing.o` and to `printf` in the C library. Running it printed `Units = 330, bill = Rs 860.00` and `Units = 256, bill = Rs 527.00`.

2.6 Compile-time, link-time and run-time errors (real examples)

Kind	When it is found	Example from my work	Message from GCC or the system
Compile time	While compiling a file. No object file is made	A missing semicolon after `int units = 330`	`err1.c:5:5: error: expected ',' or ';' before 'printf'.` After fixing it, a typing mistake appeared: `error: 'unit' undeclared (first use in this function); did you mean 'units'?`
Link time	After all files compile, when the linker joins them	Compiling `main.c` alone, without `billing.c`	`undefined reference to 'units_consumed', undefined reference to 'compute_bill', ld returned 1 exit status`
Run time	While the finished program runs	`average = total / count` when no bill was read (count = 0)	`Floating point exception`, exit status 136 (integer division by zero)

The run-time error compiles without any warning, because the compiler cannot know that `count` will be 0. It is avoided by checking `count > 0` before dividing.

**2.7 Warnings from `gcc -Wall -Wex`

A first draft of the bill function (`draft.c`) deliberately contained five typical mistakes. GCC reported:

```
draft.c:8:15: warning: comparison of integer expressions of different signedness:
   'int' and 'unsigned int' [-Wsign-compare]
draft.c:6:9: warning: unused variable 'slab_count' [-Wunused-variable]
draft.c:3:35: warning: unused parameter 'category' [-Wunused-parameter]
draft.c:17:9: warning: suggest parentheses around assignment used as truth value [-
Wparentheses] draft.c:20:22: warning: format '%d' expects argument of type 'int', but
argument 2    has type 'long int' [-Wformat=]
```

Warning	What it told me	How I resolved it
`-Wsign-compare`	`units > limit` mixes `int` and `unsigned int`, so a negative `units` would become huge	Made units `unsigned int` and gave constants a `U` suffix
`-Wunused-variable`	`slab_count` was declared but never used	Removed it
`-Wunused-parameter`	`category` was passed but not used	Removed it from `compute_bill`. The category is checked in `bill_main.c`
`-Wparentheses`	`if (total = 0)` assigns instead of comparing	Changed to `==`
`-Wformat`	`%d` used for a `long`	Used `%ld` (the Task 6 mistake)

The final programs of Parts 1 to 3 compile with **no warnings**.

3. C Program / Code

The program is the Task 5 project (`billing.h`, `billing.c`, `main.c`). The commands for every stage are in table 2.1. The deliberate error programs are the small files `err1.c`, `draft.c` and `rt.c` described above.

4. Sample Input and Output

Input: none. The outputs of each stage are shown in sections 2.2 to 2.7 (all produced by real runs).

5. Explanation of Output

The explanation of each stage's output is given with that output in section 2. In short: the preprocessor expands text (839 lines from 17), the compiler turns C into assembly with the constants already inserted as numbers, the assembler produces object files with unresolved `U` symbols, and the linker resolves them into one executable.

6. Alternative Concept and Complexity Analysis: several .c files with a shared header

6.1 What the build process is

The program is split into `input.c` (main and the readings), `tariff.c` (`units_consumed`, `compute_bill`) and `report.c` (`print_bill`), all including one shared `billing.h` (I added the prototype `void print_bill(...)`). A Makefile compiles each file separately and then links:

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra
OBJS = input.o tariff.o report.o

billing: $(OBJS)          $(CC) $(CFLAGS)
-o billing $(OBJS)

%.o: %.c billing.h
        $(CC) $(CFLAGS) -c $<

clean:            rm -f *.o billing make
```

printed these commands. First build:

```
gcc -std=c11 -Wall -Wextra -c input.c gcc -std=c11 -Wall -
Wextra -c tariff.c gcc -std=c11 -Wall -Wextra -c report.c gcc
-std=c11 -Wall -Wextra -o billing input.o tariff.o report.o
```

After `touch tariff.c` (pretending only the tariff file changed), `make` ran only two commands:

```
gcc -std=c11 -Wall -Wextra -c tariff.c
gcc -std=c11 -Wall -Wextra -o billing input.o tariff.o report.o
```

With nothing changed, it printed `make: 'billing' is up to date.` The program printed `Household 1042: 330 units, bill = Rs 860.00` and `Household 2077: 256 units, bill = Rs 527.00.`

6.2 Measured build times (average of 40 builds each; `-Wall -Wextra`, no optimisation)

Build	Commands	Time per build
Single file, full build	1 compile and link	28.7 ms
Multi-file, full build	3 compiles and 1 link	44.8 ms
Multi-file, only `tariff.c` changed	1 compile and 1 link	24.6 ms
(part) compile `tariff.c` only		10.1 ms
(part) link only		13.5 ms

6.3 Space (object files)

File	Size on disk	Machine code (text)
input.o	1,712 bytes	271 bytes
tariff.o	1,448 bytes	253 bytes
report.o	1,688 bytes	308 bytes
All three objects	4,848 bytes	832 bytes
Executable, multi-file	16,192 bytes	
Executable, single file	16,128 bytes	

6.4 Analysis

- Compile time is roughly proportional to the amount of source being compiled. With one file, **every** change recompiles the whole program, which costs O(N) for N lines. With separate files, a change to one file costs O(N_i) for its N_i lines, plus the link step, which is proportional to the total size of the object files.
- For my 74-line program the effect is **small and even reversed** for a full build: starting GCC three times (44.8 ms) is slower than once (28.7 ms), and an incremental rebuild (24.6 ms) is only about 4 ms faster than a full single-file build, because starting the compiler and the linker costs more than compiling a few lines. The advantage of separate compilation appears in large projects, where each file takes seconds to compile.
- A change to `billing.h` is **not** helped by separate compilation. In the Makefile every `.o` file depends on `billing.h`, so a tariff revision in the header recompiles all three files. This is the same cost as the single-file build.
- Separate compilation also keeps the code organised, lets several people work on different files and allows `tariff.c` to be tested alone. The cost is extra disk space for the object files (4,848 bytes here, mostly file headers and symbol tables rather than code) and a slightly larger executable (64 bytes here).

7. Evidence / Result

- The output of each stage: `gcc -E` (839 lines), `gcc -S` (assembly), `gcc -c` (object files with `nm`), and the linked executable.
- Three correctly classified errors (compile-time, link-time, run-time) and five warnings with their resolutions.
- Measured build times and object-file sizes for the multi-file alternative.

TASK 9 - IMPLEMENT AND TEST THE BILLING PROGRAM

1. Aim

To implement the complete billing program (using the constants, types, macros, header file and input checking of the earlier tasks) and to test it with normal bills, boundary cases, meter roll-over, invalid input and a set of five bills whose total I calculate by hand.

2. Explanation and Analysis

2.1 Design of the program

File	Contents
billing.h	Include guard, `meter_t`, limits, tariff constants (paise), the enum `reading_status_t` and the prototypes
billing.c	`units_consumed()` (roll-over by unsigned 16-bit arithmetic), `validate_readings()`, `status_text()`, `compute_bill()` with `#ifdef DEBUG` messages
bill_main.c	Reads one record per line: `household category previous current`. Prints each bill and a summary (accepted, rejected and the grand total in `long long` paise)
test_billing.c	Table-driven test program with expected and actual results

How a record is processed:

1. Read a line with `fgets` and convert it with `sscanf`. If `sscanf` does not return 4, the line is rejected as invalid input (letters, missing fields, `12x`).
2. Check the household number (1 to 800,000) and the category (D or C).
3. `validate_readings()` rejects a negative reading, a reading above 65535, and any pair that gives more than 10,000 units. The last check catches a current reading lower than the previous one without a valid roll-over (for example 500 to 200 gives 65,236 units).
4. Otherwise the units come from `(meter_t)(current - previous)`, which is correct across a roll-over. The bill is computed in paise and added to the `long long` grand total.

I used `fgets` plus `sscanf` here (the robust alternative of Task 6) because in a batch file one bad line must not disturb the next line.

3. C Program / Code

billing.h (final version)

```
/* billing.h - constants, types and prototypes for the Grama Jyothi billing program */
#ifndef BILLING_H          /* include guard */
#define BILLING_H typedef unsigned short meter_t;      /* 16-

bit meter register */

/* ---- Limits ---- */
#define MAX_READING        65535U    /* largest value the register can show */
#define MAX_MONTHLY_UNITS  10000U    /* plausibility limit for one household-month */
#define MAX_HOUSEHOLD_NO   800000L   /* district size */

/* ---- Tariff (all money in paise) ---- */
#define SLAB1_UNITS        100U
#define SLAB2_UNITS        200U
#define RATE1_PAISE        0L
#define RATE2_PAISE        225L
#define RATE3_PAISE        450L
#define FIXED_CHARGE_PAISE 5000L

/* ---- Result of checking a pair of readings ---- */
typedef enum {
    READING_OK = 0,
    READING_NEGATIVE,        /* a reading is below 0 */
    READING_TOO_LARGE,       /* a reading is above 65535 */     READING_IMPLAUSIBLE
/* more than MAX_MONTHLY_UNITS: bad roll-over or faulty meter */
} reading_status_t;

/* ---- Function prototypes ---- */
unsigned int units_consumed(meter_t previous, meter_t current);
```

```
reading_status_t validate_readings(long previous, long current, unsigned int
*units); const char *status_text(reading_status_t status); long
compute_bill(unsigned int units);
#endif /* BILLING_H */
```

billing.c

(final version)

```
/* billing.c - billing calculations */
#include <stdio.h>
#include "billing.h"

/* Units used, correct even after the 16-bit register rolls over (65535 -> 0) */
unsigned int units_consumed(meter_t previous, meter_t current)
{
    unsigned int units = (meter_t)(current - previous);
#ifdef DEBUG
    printf("[DEBUG] previous=%u current=%u units=%u\n",
(unsigned)previous, (unsigned)current, units);
#endif
return units; }

/* Checks both readings. Only when they are acceptable is *units set. */
reading_status_t validate_readings(long previous, long current, unsigned int *units)
{
    if (previous < 0 || current < 0) {
return READING_NEGATIVE;
    }
    if (previous > (long)MAX_READING || current > (long)MAX_READING) {
return READING_TOO_LARGE;
    }
    unsigned int used = units_consumed((meter_t)previous,
(meter_t)current);     if (used > MAX_MONTHLY_UNITS) {         return
READING_IMPLAUSIBLE;
    }
    *units = used;
return READING_OK; }

const char *status_text(reading_status_t status)
{
    switch (status) {
    case READING_OK:           return "OK";
    case READING_NEGATIVE:     return "a reading is negative";
case READING_TOO_LARGE:    return "a reading is above 65535";
    case READING_IMPLAUSIBLE:  return "current < previous without a valid roll-over, "
                                      "or more than 10000 units";
    }
    return "unknown";
}

/* Bill in paise, slab by slab */
long compute_bill(unsigned int
units)
{
    long paise = FIXED_CHARGE_PAISE;
unsigned int remaining = units;

    if (remaining > SLAB2_UNITS) {
        paise += (long)(remaining - SLAB2_UNITS) * RATE3_PAISE;
remaining = SLAB2_UNITS;
    }
    if (remaining > SLAB1_UNITS) {
        paise += (long)(remaining - SLAB1_UNITS) * RATE2_PAISE;
remaining = SLAB1_UNITS;
    }
    paise += (long)remaining * RATE1_PAISE;
#ifdef DEBUG
    printf("[DEBUG] units=%u total=%ld paise\n", units, paise);
#endif
return paise; }
```

bill_main.c

```
/* bill_main.c - batch billing: one record per line "household category previous current"
   Compile: gcc -std=c11 -Wall -Wextra -o bill bill_main.c billing.c
   Run:     ./bill < records.txt                                                    */
#include <stdio.h>
#include <string.h>
#include "billing.h"
```

```
int main(void)
{
    char line[128];     int
line_no = 0;     int accepted = 0,
rejected = 0;
    long long grand_total = 0;                /* paise, 64-bit for district scale */

    while (fgets(line, sizeof line, stdin) != NULL)
{         long household, previous, current;
char category;         unsigned int units;
line_no++;

        if (line[strspn(line, " \t\r\n")] == '\0') {     /* skip blank lines */
continue;
        }
        /* sscanf returns the number of items converted: we need all 4 */
        if (sscanf(line, " %ld %c %ld %ld", &household, &category, &previous, &current) != 4) {
printf("Line %d: REJECTED - invalid input "
                  "(expected: household category previous current)\n",
line_no);             rejected++;             continue;
        }
        if (household < 1 || household > MAX_HOUSEHOLD_NO) {
            printf("Line %d: REJECTED - household number must be 1 to
%ld\n",                    line_no, MAX_HOUSEHOLD_NO);
rejected++;             continue;
        }
        if (category != 'D' && category != 'C') {
            printf("Line %d: REJECTED - category must be D or C\n",
line_no);             rejected++;             continue;
        }
        reading_status_t status = validate_readings(previous, current, &units);
if (status != READING_OK) {
            printf("Line %d: REJECTED - %s\n", line_no,
status_text(status));             rejected++;             continue;
        }
        long total =
compute_bill(units);         grand_total
+= total;         accepted++;
        printf("Line %d: household %ld (%c): %u units, fixed Rs %ld.%02ld, "
               "energy Rs %ld.%02ld, total Rs %ld.%02ld\n",
line_no, household, category, units,
               FIXED_CHARGE_PAISE / 100, FIXED_CHARGE_PAISE % 100,
               (total - FIXED_CHARGE_PAISE) / 100, (total - FIXED_CHARGE_PAISE) % 100,
total / 100, total % 100);
    }
    printf("Accepted: %d, Rejected: %d, Grand total: Rs
%lld.%02lld\n",            accepted, rejected, grand_total / 100,
grand_total % 100);     return 0; }
```

test_billing.c

```
/* test_billing.c - table-driven tests. Compile:
   gcc -std=c11 -Wall -Wextra -o test_billing test_billing.c billing.c */
#include <stdio.h>
#include "billing.h"

typedef struct {     const
char *id;     const char
*description;     long
previous;     long current;
    reading_status_t expected_status;
    unsigned int expected_units;       /* only checked when status is READING_OK
*/     long expected_paise; } test_case_t;

static const test_case_t tests[] = {
    {"N1", "zero units",                    1000,  1000, READING_OK,           0,     5000L},
    {"N2", "50 units (free slab)",          1000,  1050, READING_OK,          50,     5000L},
    {"B1", "exactly 100 units",             1000,  1100, READING_OK,         100,     5000L},
    {"B2", "101 units",                     1000,  1101, READING_OK,         101,     5225L},
    {"N3", "150 units (slab 2)",            1000,  1150, READING_OK,         150,    16250L},
    {"B3", "exactly 200 units",             1000,  1200, READING_OK,         200,    27500L},
    {"B4", "201 units",                     1000,  1201, READING_OK,         201,    27950L},
    {"N4", "350 units (slab 3)",            1000,  1350, READING_OK,         350,    95000L},
    {"N5", "1000 units",                    1000,  2000, READING_OK,        1000,   387500L},
{"R1", "roll-over 65400 -> 120",       65400,   120, READING_OK,         256,    52700L},
    {"R2", "roll-over 65535 -> 0",         65535,     0, READING_OK,           1,     5000L},
    {"R3", "roll-over 65535 -> 99",        65535,    99, READING_OK,         100,     5000L},
    {"R4", "roll-over 65535 -> 100",       65535,   100, READING_OK,         101,     5225L},
    {"R5", "same reading 65535 -> 65535",  65535, 65535, READING_OK,           0,     5000L},
    {"L1", "exactly 10000 units",              0, 10000, READING_OK,       10000,  4437500L},
```

```
    {"I1", "negative previous reading",       -5,   100, READING_NEGATIVE,     0,        0L},
    {"I2", "negative current reading",       100,    -1, READING_NEGATIVE,     0,        0L},
    {"I3", "reading above 65535",            100, 70000, READING_TOO_LARGE,    0,        0L},
    {"I4", "current < previous, no roll-over", 500,  200, READING_IMPLAUSIBLE, 0,        0L},
{"I5", "10001 units (above monthly limit)", 0, 10001, READING_IMPLAUSIBLE, 0,        0L}
};

int main(void)
{
    int count = (int)(sizeof tests / sizeof tests[0]);
int passed = 0;

    printf("%-3s %-34s %6s %6s %7s %9s %9s  %s\n",
           "ID", "Case", "Prev", "Curr", "Units", "Expected", "Actual",
"Result");     for (int i = 0; i < count; i++) {         const test_case_t *t =
&tests[i];         unsigned int units = 0;         long paise = 0;
        reading_status_t status = validate_readings(t->previous, t->current, &units);
int ok;

        if (status == READING_OK) {
paise = compute_bill(units);
            ok = (t->expected_status == READING_OK) &&
                 (units == t->expected_units) && (paise == t->expected_paise);
        } else {
            ok = (status == t->expected_status);
        }
        if (ok) passed++;
if (status == READING_OK) {
            printf("%-3s %-34s %6ld %6ld %7u %9ld %9ld  %s\n", t->id, t->description,
                   t->previous, t->current, units, t->expected_paise, paise, ok ? "PASS" :
"FAIL");
        } else {
            printf("%-3s %-34s %6ld %6ld %7s %9s %9s  %s\n", t->id, t->description,
                   t->previous, t->current, "-", "rejected", "rejected", ok ? "PASS" : "FAIL");
        }
    }
    printf("\n%d of %d tests passed\n", passed,
count);     return (passed == count) ? 0 : 1; }
```

Compile commands (all compiled with no warnings):

- Billing program: `gcc -std=c11 -Wall -Wextra -o bill bill_main.c billing.c`
- Test program: `gcc -std=c11 -Wall -Wextra -o test_billing test_billing.c billing.c`
- Debug build: add `-DDEBUG`. For the record `1042 D 1200 1530` it additionally printed `[DEBUG] previous=1200 current=1530 units=330` and `[DEBUG] units=330 total=86000 paise`.

4. Sample Input and Output

Test 1: automated table (`./test_billing`). The expected values were worked out by hand before running (for example 201 units = 5000 + 1 × 450 + 100 × 225 = 27,950 paise). Amounts are in paise.

```
ID  Case                              Prev   Curr   Units  Expected    Actual  Result
N1  zero units                       1000   1000       0      5000      5000  PASS
N2  50 units (free slab)             1000   1050      50      5000      5000  PASS
B1  exactly 100 units                1000   1100     100      5000      5000  PASS
B2  101 units                        1000   1101     101      5225      5225  PASS
N3  150 units (slab 2)               1000   1150     150     16250     16250  PASS
B3  exactly 200 units                1000   1200     200     27500     27500  PASS
B4  201 units                        1000   1201     201     27950     27950  PASS
N4  350 units (slab 3)               1000   1350     350     95000     95000  PASS
N5  1000 units                       1000   2000    1000    387500    387500  PASS
R1  roll-over 65400 -> 120          65400    120     256     52700     52700  PASS
R2  roll-over 65535 -> 0            65535      0       1      5000      5000  PASS
R3  roll-over 65535 -> 99           65535     99     100      5000      5000  PASS
R4  roll-over 65535 -> 100          65535    100     101      5225      5225  PASS
R5  same reading 65535 -> 65535     65535  65535       0      5000      5000  PASS
L1  exactly 10000 units                 0  10000   10000   4437500   4437500  PASS
I1  negative previous reading          -5    100       -  rejected  rejected  PASS
I2  negative current reading          100     -1       -  rejected  rejected  PASS
I3  reading above 65535               100  70000       -  rejected  rejected  PASS
I4  current < previous, no roll-over  500    200       -  rejected  rejected  PASS
I5  10001 units (above monthly limit)   0  10001       -  rejected  rejected
PASS

20 of 20 tests passed
```

Test 2: record file with valid and invalid lines (`./bill < records.txt`). The input file `records.txt` was:

```
1042 D 1200 1530
2077 C 65400 120
3001 D 0 100
abc D 100 200
3002 D -5 100
3003 D 500 200
3004 D 70000 10
3005 X 100 200
3006 D 100
3007 D 12x 200
```

Output:

```
Line 1: household 1042 (D): 330 units, fixed Rs 50.00, energy Rs 810.00, total Rs 860.00
Line 2: household 2077 (C): 256 units, fixed Rs 50.00, energy Rs 477.00, total Rs 527.00
Line 3: household 3001 (D): 100 units, fixed Rs 50.00, energy Rs 0.00, total Rs 50.00
Line 4: REJECTED - invalid input (expected: household category previous current)
Line 5: REJECTED - a reading is negative
Line 6: REJECTED - current < previous without a valid roll-over, or more than 10000 units
Line 7: REJECTED - a reading is above 65535
Line 8: REJECTED - category must be D or C
Line 9: REJECTED - invalid input (expected: household category previous current)
Line 10: REJECTED - invalid input (expected: household category previous current)
Accepted: 3, Rejected: 7, Grand total: Rs 1437.00
```

Test 3: five bills with a hand-calculated total (`./bill < hand5.txt`). The input file `hand5.txt` was:

```
101 D 120 200
102 D 1000 1150
103 D 2000 2200
104 D 65450 174
105 D 500 850
```

Output:

```
Line 1: household 101 (D): 80 units, fixed Rs 50.00, energy Rs 0.00, total Rs 50.00
Line 2: household 102 (D): 150 units, fixed Rs 50.00, energy Rs 112.50, total Rs 162.50
Line 3: household 103 (D): 200 units, fixed Rs 50.00, energy Rs 225.00, total Rs 275.00
Line 4: household 104 (D): 260 units, fixed Rs 50.00, energy Rs 495.00, total Rs 545.00
Line 5: household 105 (D): 350 units, fixed Rs 50.00, energy Rs 900.00, total Rs 950.00
Accepted: 5, Rejected: 0, Grand total: Rs 1982.50
```

5. Explanation of Output

5.1 Test table, expected versus actual (every row passed: 20 of 20)

Group	Test IDs	What is tested	Result
Normal bills in each slab	N1, N2 (slab 1), N3 (slab 2), N4, N5 (slab 3)	0, 50, 150, 350 and 1000 units	Bills of Rs 50.00, 50.00, 162.50, 950.00 and 3,875.00
Boundary cases	B1 (100), B2 (101), B3 (200), B4 (201), N1 (0)	Both slab limits, one unit either side, and zero units	5000, 5225, 27500, 27950 and 5000 paise
Meter roll-over	R1 to R5	65400 to 120 (256 units), 65535 to 0 (1 unit), 65535 to 99 (100 units), 65535 to 100 (101 units), 65535 to 65535 (0 units)	All units correct, no negative bill
Plausibility limit	L1 (10,000 units accepted), I5 (10,001 rejected)	The edge of `MAX_MONTHLY_UNITS`	Accepted bill Rs 44,375.00, then rejected
Invalid readings	I1, I2 (negative), I3 (above 65535), I4 (current < previous, no roll-over)	Range and roll-over checks	All rejected

Sample hand calculation (test N5, 1000 units): 5000 + 100 × 225 + 800 × 450 = 5000 + 22,500 + 360,000 = 387,500 paise = Rs 3,875.00.

5.2 Record file test, expected versus actual

Line	Input	Expected	Actual	Match
1	1042 D 1200 1530	330 units: 5000 + 130 × 450 + 100 × 225 = 86,000 paise = Rs 860.00	Rs 860.00	Yes
Line	Input	Expected	Actual	Match

2	2077 C 65400 120	Roll-over, 256 units: 5000 + 56 × 450 + 100 × 225 = 52,700 paise = Rs 527.00	Rs 527.00	Yes
3	3001 D 0 100	100 units, free slab: Rs 50.00	Rs 50.00	Yes
4	abc D 100 200	Letters: rejected, `sscanf` did not return 4	Rejected, invalid input	Yes
5	3002 D -5 100	Negative reading: rejected	Rejected	Yes
6	3003 D 500 200	No valid roll-over (65,236 units): rejected	Rejected	Yes
7	3004 D 70000 10	Above 65535: rejected	Rejected	Yes
8	3005 X 100 200	Category X: rejected	Rejected	Yes
9	3006 D 100	Missing field: rejected	Rejected, invalid input	Yes
10	3007 D 12x 200	Digits followed by a letter: rejected	Rejected, invalid input	Yes

Summary line: 3 accepted, 7 rejected, grand total Rs 860.00 + 527.00 + 50.00 = **Rs 1,437.00**, which matches the program output.

5.3 Five bills with a hand-calculated total

Household	Readings	Units	Hand calculation (paise)	Bill
101	120 to 200	80	5000 + 0	5,000 = Rs 50.00
102	1000 to 1150	150	5000 + 50 × 225 = 16,250	Rs 162.50
103	2000 to 2200	200	5000 + 100 × 225 = 27,500	Rs 275.00
104	65450 to 174 (roll-over)	86 + 174 = 260	5000 + 60 × 450 + 100 × 225 = 5000 + 27,000 + 22,500 = 54,500	Rs 545.00
105	500 to 850	350	5000 + 150 × 450 + 100 × 225 = 5000 + 67,500 + 22,500 = 95,000	Rs 950.00
Total			5,000 + 16,250 + 27,500 + 54,500 + 95,000 = **198,250 paise**	**Rs 1,982.50**

The program printed `Grand total: Rs 1982.50`, which agrees exactly with the hand calculation. For household 104, the units 260 are 86 (from 65450 up to the roll-over at 65536) plus 174.

6. Alternative Concept and Complexity Analysis: slab arrays and a loop

6.1 What the solution is

The slab limits and rates are stored in two arrays, and one loop handles every slab:

```
static const unsigned int slab_limit[] = {100U, 200U, UINT_MAX};
static const long          slab_rate[]  = {0L,    225L,  450L};
#define SLAB_COUNT (sizeof slab_limit / sizeof slab_limit[0])

long compute_bill_loop(unsigned int units)
{
    long paise = FIXED_CHARGE_PAISE;
unsigned int lower = 0;
    for (size_t s = 0; s < SLAB_COUNT && units > lower; s++) {
unsigned int upper = slab_limit[s];
        unsigned int in_slab = ((units < upper) ? units : upper) -
lower;         paise += (long)in_slab * slab_rate[s];         lower =
upper;
    }
    return paise;
}
```

The last limit is `UINT_MAX`, which means "no upper limit". I compared this function with `compute_bill()` for every value from 0 to 10,000 units: **0 differences**.

6.2 Time and space (s = number of slabs)

Property	Fixed if statements	Arrays and a loop
Time per bill	A fixed number of tests (2 here), so O(1)	O(s): one pass for each slab used
Time, 100 million bills, 3 slabs (-O0)	0.316 s	0.597 s
Extra space	None	Two arrays: 3 × 4 + 3 × 8 = **36 bytes**
Adding a slab	Write a new `if` block and change the code	Add one value to each array, the loop code stays the same

Measured growth with s (20 million bills each, all slabs used, -O0): s = 3: 0.127 s, s = 12: 0.601 s, s = 48: 1.805 s. The time grows with s (about 4.7 times for 4 times more slabs, and 14 times for 16 times more), which confirms O(s). A real tariff has only a handful of slabs, so the loop is still fast, and 0.6 s for 100 million bills is about 6 nanoseconds per bill.

Which is easier to change? The array design. A tariff revision with more slabs, or different limits, changes only the table data. The `if` design needs new code for each new slab, which is more error-prone. The `if` design is slightly faster and is easy for a beginner to read when there are only three slabs. The array design also fits well with Task 11, where the tariff can be loaded from a file into the same arrays.

7. Evidence / Result

- Working source code (section 3), compiled without warnings.
- A test table with expected and actual results for every case: 20 of 20 automated tests passed, and all 10 record-file lines behaved as expected.
- The five-bill hand calculation: Rs 1,982.50 by hand and by program.

TASK 10 - COMPARE EXPECTATIONS WITH EXECUTION

1. Aim

To compare what I expected from my type and constant design with what actually happened: whether float and integer-paise totals agree at 4,000, 60,000 and 800,000 bills, whether `int`, `long` and `long long` totals overflow at district scale, and how `sizeof` and the ranges on my system compare with the minimum sizes guaranteed by the C standard.

2. Explanation and Analysis

2.1 What I expected before running the experiments

#	Expectation	Reason
1	Integer paise totals will be exact at all three scales	Integer addition has no rounding
2	`float` rupee totals will drift from the exact total, and badly once the total passes about 16.7 million	A `float` has a 24-bit mantissa, so above 2^{24} = 16,777,216 it cannot store every whole number
3	`int` (32 bits, maximum 2,147,483,647 paise = Rs 21,474,836.47) will overflow at district scale. Taluk scale will fit, because my Task 1 estimate was 60,000 × 16,250 = 975,000,000	Task 1 calculation
4	`long long` will not overflow at any scale	Maximum 9.22×10^{18}
5	`long` may be 4 or 8 bytes, depending on the system	The C standard guarantees only 32 bits

2.2 Test design

Two sets of bills were used. **Dataset A:** every bill is Rs 162.35 (the same test as Task 2). **Dataset B:** the units of bill number i are `(i * 37) % 400`, so the bills are spread over all three slabs, using the real tariff. For each set I added n = 4,000, 60,000 and 800,000 bills in six accumulators: `int`, `long`, `long long` (all in paise), and `float` and `double` (in rupees). The `float` and `double` totals were converted to paise once at the end. The integer overflow was detected with GCC's `__builtin_add_overflow`.

I also ran the real billing program (`bill`, from Task 9) on three record files of 4,000, 60,000 and 800,000 households. In each file every tenth household had a meter roll-over (previous 65500), so the roll-over rule was used at every scale. An independent Python calculation gave the expected totals.

2.3 Measured results: float and double against integer paise

Dataset	n (bills)	Exact total (integer paise)	float: difference	double: difference
A (Rs 162.35 each)	4,000	Rs 649,400.00	**+500 paise (Rs 5.00)**	0
A	60,000	Rs 9,741,000.00	**-11,500 paise (Rs 115.00)**	0
A	800,000	Rs 129,880,000.00	**-62,672,000 paise (Rs 626,720.00)**	0
B (units spread over slabs)	4,000	Rs 1,656,875.00	0	0
Dataset	**n (bills)**	**Exact total (integer paise)**	**float: difference**	**double: difference**
B	60,000	Rs 24,853,125.00	**-266,700 paise (Rs 2,667.00)**	0
B	800,000	Rs 331,375,000.00	**+29,156,000 paise (Rs 291,560.00)**	0

The `double` totals agreed with the integer totals in every case, but only after the single final rounding. The `float` totals agreed only at 4,000 bills in dataset B. This matches Part 1 (Task 2) and Part 3 (Task 7), where the same numbers were found.

2.4 Measured results: integer overflow

Dataset	n	Exact total (paise)	`int` (32 bits)	`long` (8 bytes here)	`long long`
A	4,000	64,940,000	fits	fits	fits
A	60,000	974,100,000	fits (45% of the range)	fits	fits
A	800,000	12,988,000,000	**overflows at bill 132,275**, wraps to 103,098,112	fits	fits
B	4,000	165,687,500	fits	fits	fits

B	60,000	2,485,312,500	**overflows at bill 51,848**, wraps to -1,809,654,796	fits	fits
B	800,000	33,137,500,000	**overflows at bill 51,848**, wraps to -1,222,238,368	fits	fits

Check of the first overflow point for dataset A: 2,147,483,647 / 16,235 = 132,274.9, so 132,274 bills still fit (132,274 × 16,235 = 2,147,468,390) and the 132,275th bill makes the total pass the limit (132,275 × 16,235 = 2,147,484,625), exactly as measured.

2.5 Measured results: the real billing program (`./bill < records` with the Task 9 program)

Households	Grand total printed by the program	Python expected total	Run time (output to a file)
4,000	Rs 1,656,875.00	Rs 1,656,875.00	0.004 s
60,000	Rs 24,853,125.00	Rs 24,853,125.00	0.038 s
800,000	Rs 331,375,000.00	Rs 331,375,000.00	0.49 s

All 800,000 records were accepted (80,000 households started at reading 65500, and most of them rolled over), the totals equal the independent calculation to the paisa, and they equal the exact totals of dataset B in table 2.3.

2.6 sizeof and ranges compared with the C standard

Type	Minimum guaranteed by the C standard	On my system (measured)	Comment
char	at least 8 bits	1 byte, CHAR_BIT = 8	equal to the minimum
short	at least 16 bits (SHRT_MAX at least 32,767)	2 bytes, SHRT_MAX = 32,767	equal
int	at least 16 bits (INT_MAX at least 32,767)	4 bytes, INT_MAX = 2,147,483,647	**larger than the minimum**
long	at least 32 bits (LONG_MAX at least 2,147,483,647)	**8 bytes**, LONG_MAX = 9,223,372,036,854,775,807	**larger than the minimum** (it is 4 bytes on 64-bit Windows)
long long	at least 64 bits	8 bytes, LLONG_MAX = 9,223,372,036,854,775,807	equal
float	FLT_DIG at least 6	4 bytes, FLT_DIG = 6, 24-bit mantissa	equal
double	DBL_DIG at least 10	8 bytes, DBL_DIG = 15, 53-bit mantissa	**better than the minimum**

2.7 Where reality differed from my expectation, and the reasons

1. `int` **overflowed at taluk scale for dataset B**, although I expected taluk to fit. The estimate in Task 1 used a typical 150-unit bill (Rs 162.50). Dataset B has an average bill of about Rs 414 (Rs 24,853,125 / 60,000), so the total passes the `int` limit after only 51,848 bills. *Reason:* the safe size of a type depends on the largest realistic data, not on a typical bill. Expectation 3 was right for dataset A (taluk fits, district overflows) and wrong for dataset B. This is why the recommended total is `long long` for every scale.
2. `long` **was 8 bytes**, not the 4 bytes of the guaranteed minimum. The `long` total therefore did not overflow here. *Reason:* the standard fixes only minimum sizes. Linux on 64-bit PCs uses 8 bytes, while 64-bit Windows keeps `long` at 4 bytes. On Windows the `long` column would overflow exactly like `int`. A portable program must use `long long` for totals. Bills (at most 4,437,500 paise) fit in a 32-bit `long`, so `long` is safe for a single bill.
3. `float` **was exactly right for dataset B at 4,000 bills.** *Reason:* Rs 2.25 and Rs 4.50 are multiples of 0.25, which binary can store exactly, and the total (about Rs 1.66 million) stays below the point where quarter-rupees stop being representable. It is a coincidence. With Rs 162.35 (dataset A), the error was Rs 5.00 already at 4,000 bills.
4. `int` **is guaranteed to have only 16 bits.** A household number up to 800,000 does not fit in a 16-bit `int` (maximum 32,767). On my system `int` has 32 bits, so it works, but only `long` (at least 32 bits) is guaranteed to hold it. The bill program `bill_main.c` already reads the household number into a `long`.
5. **Integer paise and** `double` **rupees agreed**, as expected for double after rounding, but this agreement is not guaranteed (see section 6).

3. C Program / Code

```
/* task10.c - compares float, double and integer totals at three scales.
   Compile: gcc -std=c11 -Wall -Wextra -o task10 task10.c                      */
#include <stdio.h>
#include <limits.h>
#include <float.h>

/* Bill in paise for the real tariff (Rs 50 fixed, 100 free, Rs 2.25, Rs 4.50) */
static long bill_paise(unsigned int units)
{
    long p = 5000L;
    if (units > 200U) { p += (long)(units - 200U) * 450L; units = 200U;
}     if (units > 100U) { p += (long)(units - 100U) * 225L; }
return p; }

static void run(const char *name, int n, int same_bill)
{
    int   total_int = 0;          /* 32-bit: wraps when it overflows            */
long  total_long = 0;         /* 8 bytes here, but only 4 on Windows        */
long long total_ll = 0;       /* at least 64 bits everywhere                */
float  total_f = 0.0f;        /* rupees as float                            */
double total_d = 0.0;         /* rupees as double                           */
int first_overflow = 0;       /* number of the bill at which int overflowed */

    for (int i = 0; i < n; i++) {
        long bill = same_bill ? 16235L : bill_paise((unsigned int)((i * 37L) % 400));
if (__builtin_add_overflow(total_int, (int)bill, &total_int) && first_overflow == 0) {
first_overflow = i + 1;
        }
        total_long += bill;
total_ll   += bill;
        total_f    += (float)bill / 100.0f;
total_d    += (double)bill / 100.0;
    }
    long long pf = (long long)((double)total_f * 100.0 + 0.5);
long long pd = (long long)(total_d * 100.0 + 0.5);

    printf("%s, n = %d\n", name, n);
    printf("  long long total : Rs %lld.%02lld (exact)\n", total_ll / 100, total_ll %
100);     printf("  long total      : Rs %ld.%02ld\n", total_long / 100, total_long %
100);     if (first_overflow == 0) {
        printf("  int total       : %d paise, no overflow\n", total_int);
    } else {
        printf("  int total       : %d paise (WRAPPED, overflow at bill %d)\n",
total_int, first_overflow);
    }
    printf("  float rupees    : Rs %lld.%02lld, difference from exact %+lld
paise\n",            pf / 100, pf % 100, pf - total_ll);
    printf("  double rupees   : Rs %lld.%02lld, difference from exact %+lld
paise\n",            pd / 100, pd % 100, pd - total_ll); }

int main(void)
{
    printf("char %zu, short %zu, int %zu, long %zu, long long %zu, float %zu, double %zu
bytes\n",            sizeof(char), sizeof(short), sizeof(int), sizeof(long), sizeof(long long),
sizeof(float), sizeof(double));
    printf("CHAR_BIT=%d SHRT_MAX=%d INT_MAX=%d LONG_MAX=%ld LLONG_MAX=%lld\n",
           CHAR_BIT, SHRT_MAX, INT_MAX, LONG_MAX, LLONG_MAX);
    printf("FLT_DIG=%d FLT_MANT_DIG=%d DBL_DIG=%d DBL_MANT_DIG=%d\n\n",
FLT_DIG, FLT_MANT_DIG, DBL_DIG, DBL_MANT_DIG);

    int sizes[3] = {4000, 60000, 800000};
    for (int k = 0; k < 3; k++) run("Dataset A: every bill Rs 162.35", sizes[k], 1);
printf("\n");
    for (int k = 0; k < 3; k++) run("Dataset B: units = (i*37) % 400", sizes[k],
0);     return 0; }
```

Compile: `gcc -std=c11 -Wall -Wextra -o task10 task10.c` (no warnings). The program uses `__builtin_add_overflow`, a GCC feature that reports whether an addition overflowed, so the overflow point could be measured without relying on undefined behaviour.

4. Sample Input and Output

Input: none for `task10`. For the billing program the input files were `rec_4000.txt`, `rec_60000.txt` and `rec_800000.txt` (one line per household, for example `1 D 65500 65500`, `11 D 65500 334`, `12 D 1000 1007`).

Output of `task10` (**produced by running it with GCC; verify on your machine**):

```
char 1, short 2, int 4, long 8, long long 8, float 4, double 8 bytes
CHAR_BIT=8 SHRT_MAX=32767 INT_MAX=2147483647 LONG_MAX=9223372036854775807
    LLONG_MAX=9223372036854775807
FLT_DIG=6 FLT_MANT_DIG=24 DBL_DIG=15 DBL_MANT_DIG=53

Dataset A: every bill Rs 162.35, n = 4000
long long total : Rs 649400.00 (exact)
long total      : Rs 649400.00
  int total       : 64940000 paise, no overflow
  float rupees    : Rs 649405.00, difference from exact +500 paise
double rupees   : Rs 649400.00, difference from exact +0 paise
Dataset A: every bill Rs 162.35, n = 60000
long long total : Rs 9741000.00 (exact)
long total      : Rs 9741000.00
  int total       : 974100000 paise, no overflow
  float rupees    : Rs 9740885.00, difference from exact -11500 paise
double rupees   : Rs 9741000.00, difference from exact +0 paise
Dataset A: every bill Rs 162.35, n = 800000
long long total : Rs 129880000.00 (exact)
long total      : Rs 129880000.00
  int total       : 103098112 paise (WRAPPED, overflow at bill 132275)
float rupees    : Rs 129253280.00, difference from exact -62672000 paise
double rupees   : Rs 129880000.00, difference from exact +0 paise

Dataset B: units = (i*37) % 400, n = 4000
long long total : Rs 1656875.00 (exact)
long total      : Rs 1656875.00
  int total       : 165687500 paise, no overflow
  float rupees    : Rs 1656875.00, difference from exact +0 paise
double rupees   : Rs 1656875.00, difference from exact +0 paise
Dataset B: units = (i*37) % 400, n = 60000
long long total : Rs 24853125.00 (exact)
long total      : Rs 24853125.00
  int total       : -1809654796 paise (WRAPPED, overflow at bill 51848)
float rupees    : Rs 24850458.00, difference from exact -266700 paise
double rupees   : Rs 24853125.00, difference from exact +0 paise
Dataset B: units = (i*37) % 400, n = 800000
long long total : Rs 331375000.00 (exact)
long total      : Rs 331375000.00
  int total       : -1222238368 paise (WRAPPED, overflow at bill 51848)
float rupees    : Rs 331666560.00, difference from exact +29156000 paise
double rupees   : Rs 331375000.00, difference from exact +0 paise
```

Last line of the real billing program for each file (`./bill < rec_4000.txt` and so on):

```
Accepted: 4000, Rejected: 0, Grand total: Rs 1656875.00
Accepted: 60000, Rejected: 0, Grand total: Rs 24853125.00
Accepted: 800000, Rejected: 0, Grand total: Rs 331375000.00
```

5. Explanation of Output

- **Dataset A, n = 4,000:** 4,000 × 16,235 = 64,940,000 paise = Rs 649,400.00. The `float` total is Rs 649,405.00, which is +500 paise (Rs 5.00) wrong. The `float` cannot store 162.35 exactly and the error adds up.
- **Dataset A, n = 800,000:** the exact total is 800,000 × 16,235 = 12,988,000,000 paise. The `int` accumulator overflowed at bill 132,275 and finished at 103,098,112 (12,988,000,000 - 3 × 4,294,967,296 = 103,098,112), which is a silent, wrong answer.
- **Dataset B, n = 60,000:** the total of 2,485,312,500 paise is larger than 2,147,483,647. The wrapped `int` value is 2,485,312,500 - 4,294,967,296 = -1,809,654,796, a **negative** total.
- `float` **at 800,000 bills:** once the running total passes 16.7 million rupees, each added bill is rounded to a spacing of 1 or more, so thousands of bills are rounded the wrong way and the total ends up Rs 291,560.00 too high (dataset B) or Rs 626,720.00 too low (dataset A).
- `double`**:** its error stays far below half a paisa (about 10^{-3} rupees at 800,000 bills, Task 2), so the single final rounding hides it.
- **The billing program** reached exactly the same totals as dataset B, through a different path (reading 800,000 text records, validating them, applying roll-over, and computing each bill), which also cross-checks the integer total.

6. Alternative Concept and Complexity Analysis: district total accumulated in a double

6.1 What the solution is

Instead of `long long total_paise`, the district total is kept in `double total`. There are two ways to do it.

- **Rupees with fractions** (`total += bill / 100.0`): this is dataset A/B above. The error is zero at 800,000 bills here only after the final rounding and is not guaranteed.
- **Whole paise held in a double** (`total += (double)bill_paise`): all values are whole numbers, so a `double` can store them exactly, up to a limit.

6.2 Measurements (program `alt10.c`)

```
1. paise in long long = 12988000000, in double = 12988000000, difference = 0
2. 2^53 = 9007199254740992, 2^53 + 1 = 9007199254740992 (equal: yes)
3. LLONG_MAX = 9223372036854775807, LLONG_MAX - 5 + 10 overflows: yes (wrapped to -
   9223372036854775804)
4. 300M additions: long long 0.730 s, double 0.858 s (1050000000 1050000000)
   sizeof(long long) = 8, sizeof(double) = 8
```

6.3 When does each type lose accuracy?

Property	`long long` (paise)	`double` (whole paise)	`double` (rupees with fractions)
At 800,000 bills (Rs 331 million)	exact	exact (difference 0)	correct only after final rounding
Where accuracy is lost	**Overflow** at 9,223,372,036,854,775,807 paise (Rs 9.2×10^{16}). The result wraps silently to a negative number	**Loss of precision** above 2^{53} = 9,007,199,254,740,992 paise (Rs 9.0×10^{13}, about Rs 90 trillion). Odd whole numbers can no longer be stored, so additions are silently lost	At all scales, as shown: Rs 5.00 off at 4,000 bills with `float`, and drift with `double`
Kind of failure	Sudden, obvious wrong value (huge negative)	Gradual, silent, looks plausible	Gradual and silent
Time per addition	O(1), 0.73 s for 300 million additions	O(1), 0.86 s for 300 million additions	O(1)
Space per value	8 bytes	8 bytes	8 bytes

Is either safe at district scale? My largest district totals are: normal data Rs 331 million (3.3×10^{10} paise); worst case with the 65,535-unit rule 800,000 × 29,428,250 = 2.35×10^{13} paise; worst case with the 10,000-unit limit 800,000 × 4,437,500 = 3.55×10^{12} paise; and the smart-meter design of Task 12, at most 7.57×10^{13} paise. All are below the `double` limit of 9.0×10^{15} by a factor of at least 119, and below the `long long` limit of 9.2×10^{18} by a factor of at least 121,000. So a `double` holding whole paise would give exact answers at district scale.

I still prefer `long long`: (1) it is exact by construction and needs no analysis of limits; (2) its range is about 1,000 times larger; (3) the remainder operator `%` for splitting rupees and paise works only on integers (a `double` would need `fmod`); (4) a reader of the program sees clearly that money is a whole number. The time difference is small (0.73 s against 0.86 s for 300 million additions) and the space is the same.

7. Evidence / Result

- Measured totals at three scales (tables 2.3 to 2.5), with the real program's totals equal to an independent calculation.
- Overflow points: `int` at bill 132,275 (dataset A) and bill 51,848 (dataset B); `long long` never overflows.
- A `sizeof` and range table against the C standard minimums (2.6), and the reasons for each difference (2.7).

TASK 11 - RECOMMEND AND DEFEND THE FINAL DESIGN

1. Aim

To recommend the final data types, constants and preprocessor features for the co-operative, to explain how a tariff revision is carried out, to state the limitations, and to defend the design at district scale using the evidence of my own experiments in Tasks 1 to 10.

2. Explanation and Analysis 2.1

Recommended data types

Data item	Recommended type	Why, and the evidence
Household number (1 to 800,000)	`long`	The standard guarantees `long` at least 32 bits. `int` is guaranteed only 16 bits (max 32,767), which is too small (Task 10). `unsigned int` is also correct on every modern system
Category	`char`	One letter, `'D'` or `'C'`
Meter reading	`meter_t` = `unsigned short`	Matches the 16-bit register (0 to 65,535). Wrap-around gives the roll-over rule: 65400 to 120 gives 256 units (Task 2)
Units consumed	`unsigned int`	Never negative and at most 10,000 in my plausibility rule. Any unsigned int has at least 65,535 as its maximum
Fixed charge, energy charge and one bill	`long`, in **paise**	Largest bill 4,437,500 paise at 10,000 units, far below the guaranteed 2,147,483,647 of a 32-bit `long`. Float rupees gave errors of Rs 5.00 at 4,000 bills (Tasks 2 and 10)
District and taluk totals	`long long`, in **paise**	`int` overflowed at bill 51,848 (dataset B) and bill 132,275 (dataset A); `long` is only 32 bits on Windows. `long long` holds 9.22×10^{18}, and the worst-case district total is 2.35×10^{13} (Task 10)
Money, never	`float` or `double` rupees	Measured errors up to Rs 626,720 at 800,000 bills (Task 10)

2.2 Recommended constants and preprocessor features

Feature	Recommendation	Evidence
`#define`	For tariff values and limits, in **one header** (`billing.h`), named with units (`RATE2_PAISE`, `SLAB1_UNITS`) and type suffixes (`200U`, `5000L`)	Task 3: changing `RATE2_PAISE` from 225 to 230 changed every slab-2 bill consistently (Rs 162.50 to Rs 165.00 for 150 units)
`enum`	For the result of checking readings (`reading_status_t`)	Task 9: the status code made every rejection testable
`const`	For typed values that need type checking or a debugger name	Task 3 comparison table
Macros	Only small, fully parenthesised ones. Prefer `static inline` functions	Task 4: `BAD_SQUARE(a + 1)` gave 9 instead of 25, and `MAX(i++, j)` changed `i` twice

Include guards	In every header	Task 5
`#ifdef DEBUG`	For intermediate values, with `gcc -DDEBUG`. It adds no code or time to the release build	Task 5: the release executable was 2,024 bytes of code against 2,381 with a run-time debug flag
Compiler flags	Always `-std=c11 -Wall -Wextra`, with warnings treated as bugs	Task 8: five typical mistakes were all caught by these flags

2.3 How a tariff revision will be carried out (compiled-in design of Tasks 3 to 9)

1. The board gives the new values in writing, with the date from which they apply.
2. Only the matching `#define` lines in `billing.h` are edited (for example `#define RATE2_PAISE 230`). Nothing else in any file may contain a tariff number.
3. The program is recompiled with `gcc -std=c11 -Wall -Wextra`, and any warning is fixed.
4. The test program `test_billing` is run. Its expected values for the changed slab are updated by hand first (for example 150 units: 5000 + 50 × 230 = 16,500 paise), so the test checks the new tariff independently.
5. The program is run on the five-bill hand-checked file and a sample of real records.
6. The new executable is copied to every office and the old one is removed.

Step 6 is the weak point: every revision needs a recompile and a redistribution. The alternative concept below removes it.

2.4 Limitations of my design

1. The tariff is compiled in, so every revision needs a rebuild and redistribution (addressed in section 6).
2. Meter readings are typed or collected by hand, once a month. Readings that are wrong but plausible cannot be detected.
3. The rule "at most 10,000 units a month" is my assumption (Task 6). A real consumer above that limit would be rejected, and one who uses less but has a meter fault would not be caught.
4. At most one roll-over a month is assumed. A meter with a full cycle of 65,536 units or more in a month would give a wrong result.
5. One tariff is used for all categories. The category letter is stored and checked but does not change the bill.
6. Input is plain text, line by line. There is no database, no history and no duplicate-household detection.
7. No taxes, discounts, arrears or late fees are included.
8. The design depends on `long long` (a C99 feature), on a compiler that gives 16-bit `unsigned short`, and on `-std=c11`.
9. Smart-meter readings and time-of-day tariffs are not covered by this design (Task 12 deals with this).

2.5 Does the design remain correct at district scale? Yes, for the monthly-reading requirement, and the evidence is measured.

- The real program processed 800,000 records in 0.49 s and printed Rs 331,375,000.00, which is exactly the independent total (Task 10).
- Totals are kept in `long long`. The worst-case district total of 800,000 × Rs 294,282.50 (largest bill with 65,535 units) is 2.35×10^{13} paise, which is 390,000 times below the `long long` maximum. With the 10,000-unit limit it is 3.55×10^{12} paise, which is 2.6 million times below.
- `int` totals failed at 51,848 to 132,275 bills, so `int` is rejected even for a taluk.
- Float rupees failed at all scales and are rejected.
- The roll-over rule was exercised at every scale (every tenth household in the 800,000-record test). **2.6 What the experiments say about each decision**

Decision	Experiment	Result
Money in integer paise	Tasks 2, 7, 10	float errors: Rs 5.00 (4,000 bills) to Rs 626,720.00 (800,000 bills)
16-bit unsigned subtraction for roll-over	Tasks 2, 9	65400 to 120 gives 256 units; 5 roll-over tests passed
`long long` totals	Task 10	`int` overflowed; `long` is 32 bits on Windows
One header for the tariff	Task 3	225 to 230 changed all bills consistently
Validation before billing	Tasks 6, 9	20 of 20 tests passed, including letters and negative readings
Warnings enabled	Task 8	5 of 5 deliberate mistakes caught

3. C Program / Code

The final program is the Task 9 program (`billing.h`, `billing.c`, `bill_main.c`). Its compile command is `gcc -std=c11 -Wall -Wextra -o bill bill_main.c billing.c`. For the alternative concept, the tariff is loaded by the function `load_tariff()` that is listed completely in Task 12. Its method is:

```
open the tariff file
for each line:
    skip blank lines and lines starting with #
read  key = number
    find the key in a table (fixed_charge_paise, slab1_units, ...
peak_extra_percent)     check the number against that key's minimum and maximum,
and store it after the last line:
    report every key that is missing
    check that slab1_units <= slab2_units and peak_start_hour < peak_end_hour
return "valid" only if no error was found
```

4. Sample Input and Output

Example of a revision with the configuration file, using the Task 12 program and test data (330 kWh used entirely in the peak hours). The revised file changed `rate2_paise` from 225 to 230 and `peak_extra_percent` from 20 to 25. No recompiling was done.

```
Original tariff.cfg  : total Rs 1022.00
Revised tariff_rev.cfg: total Rs 1068.75
```

Error messages when the tariff file is wrong (real runs):

```
Cannot open tariff file 'nofile.cfg'
Tariff file: missing value for rate2_paise
Tariff file line 13: unknown key 'peak_extra_pct'
Tariff file line 9: rate3_paise must be 0 to 100000
Tariff file line 5: expected  key = number
```

5. Explanation of Output

- The Rs 1,022.00 and Rs 1,068.75 totals are explained in Task 12 (section 5). The revision changed two numbers in a text file, and the same executable gave the new bill.
- Each wrong tariff file stopped the program with a clear message (and exit status 1) instead of producing wrong bills. A missing file, a missing key, a misspelt key, a negative or out-of-range rate and a non-numeric value were all caught.

6. Alternative Concept and Complexity Analysis: tariff read from a configuration file at start-up

6.1 What the solution is

The tariff values are no longer `#define` lines. They are stored in a structure `tariff_t` (9 values of type `long long`) that the program fills from `tariff.cfg` when it starts. A revision is an edit of the text file. The slab code uses `t->rate2_paise` instead of `RATE2_PAISE`. Limits that never change (paise per rupee, 16-bit register size, units per kWh) stay as `#define`.

6.2 Cost of the start-up read (measured by calling the real `load_tariff()` 100,000 times; the operating system had the 711-byte file in memory)

Measurement	Result
Time to read and check t = 9 tariff values	**about 4.0 microseconds** (3.96 and 4.07 in two runs), about 0.44 microseconds per value
Time to start the whole program, read the file and exit	about 0.75 milliseconds
Space for the values in memory	72 bytes (`sizeof(tariff_t)`)
Space of the file on disk	711 bytes (with comments)
Space for a `#define` constant	none (it is a number inside the instructions)

6.3 Cost of the alternative: recompiling and redistributing

Step	Compiled-in tariff	Tariff in a configuration file
Work for one revision	Edit the header, recompile, retest, copy the executable to every office	Edit one text file and copy it to every office
Compile time (measured)	about 110 ms for the 285-line program with `-O2`	none
Start-up time per run	0	0.004 ms (4 microseconds)

Complexity	Recompile: O(N) for N lines of source, plus redistribution to every office	Load: O(t) for t tariff values, executed once per run
Risk	An office may keep running an old executable (partly revised tariff, as in the spreadsheet)	A mistyped value is possible, but `load_tariff()` checks every value and refuses to run if one is wrong
Needs a compiler at the office	Yes, or a new executable for every revision	No

6.4 Conclusion. The start-up read costs about 4 microseconds for 9 values, which is negligible compared with the 110 ms (plus the human work and risk) of recompiling and redistributing. The configuration file is the better design for a tariff that the board changes. The price is 72 bytes of memory, the need for validation code (about 60 lines), and one more file that must be kept correct. This is the design adopted in Task 12.

7. Evidence / Result

- A recommendation of types, constants and preprocessor features, each tied to a measured result (tables 2.1, 2.2 and 2.6).
- A tariff revision procedure and its measured example (section 2.3 and 4).
- Limitations and a district-scale check: 800,000 records in 0.49 s with an exact total (2.4 and 2.5).

TASK 12 - RE-EVALUATE AND ADAPT THE DESIGN

Decision in one line

The original design should be **MODIFIED**, not kept as it is and not thrown away: the **ideas** (integer paise, round once, validate first, `long long` totals, single source of tariff values, warnings-clean C) are all kept, but the **data types, constants and input handling** are changed (wider integers, tariff loaded from a file, streaming time-stamped readings, a peak-hour rule). The reasons and the calculations follow in sections A to D.

Assumptions for Task 12

1. A reading is the **energy used in that 15-minute period, in watt-hours (Wh)**. It is not a running total. The time stamp is the **start** of the period.
2. **Peak hours** are 6 PM to 10 PM, so a period that starts at 18:00, 18:15, ... or 21:45 is a peak period (16 periods a day). The period that starts at 22:00 is off-peak.
3. The 20% extra is added to the price of **each unit used in peak periods** (slab 1 stays free; slab 2 becomes Rs 2.25 × 1.20 = Rs 2.70; slab 3 becomes Rs 4.50 × 1.20 = Rs 5.40). The Rs 50 fixed charge is not changed.
4. The monthly slab limits (100 and 200 units, where 1 unit = 1 kWh = 1,000 Wh) apply to the energy of the calendar month, counted **in time order**. A reading that crosses a slab limit is split between the two slabs at Wh precision. (The board should confirm this rule.)
5. A year has 365 days (35,040 = 365 × 96 readings), and the bill is made once a month.
6. The largest supply is **20 kW**, so one reading is at most 20,000 W × 0.25 h = **5,000 Wh**. This is my assumption and is checked by the program.
7. The readings come as text lines `household month day hour minute wh`, sorted by household and month.
8. No taxes, one tariff for all categories (as in the earlier tasks).

A. IMPACT ANALYSIS (2 marks)

A.1 What has changed from the original problem

Topic	Original problem	Changed requirement	Effect on the design
Reading frequency	1 reading a month (a pair of readings per bill)	A reading every 15 minutes: 96 a day, 2,976 in a 31-day month, **35,040 a year**	Data volume grows by about 2,900 times (12 readings a year to 35,040)
Unit of reading	Whole units (kWh) on a 16-bit register	**Watt-hours** (1 kWh = 1,000 Wh)	Numbers are 1,000 times larger, and a conversion between Wh and kWh is needed
What is read	Cumulative register that can roll over at 65,535	Energy used in each 15 minutes	No roll-over rule is needed, but each reading needs a time stamp
Tariff	Slabs only	Slabs **and** a time-of-day rule (+20% from 6 PM to 10 PM)	Each reading must be priced by its hour
Tariff changes	A recompile of `billing.h`	Board changes values **without recompiling**	Values must be read from a file when the program starts
Money	Whole paise from whole units	Fractions of a paisa now appear (Wh × paise per kWh ÷ 1,000, and the 20%)	A rounding rule is now really needed (Task 7)
Data volume to process	800,000 bills a month	800,000 × 35,040 = **28,032,000,000 readings a year**	Readings cannot be stored in memory. They must be processed one at a time

A.2 Which data types are no longer sufficient (all values are calculated in section B)

Data item	Original type and limit	Largest value now	Verdict
Energy of one household in a year, in Wh	`meter_t`, 65,535	175,200,000 Wh (2,673 times more)	Not sufficient
Energy of the district in a year, in Wh	`unsigned int`, 4,294,967,295	140,160,000,000,000 Wh	Not sufficient. Needs 47 bits

Number of readings in a year (district)	`unsigned int`	28,032,000,000	Not sufficient. Needs 35 bits
Unrounded charge (Wh × rate × percent) for a month	`long` (32 bits on some systems)	803,520,000,000	Not sufficient for a 32-bit `long`

Data item	Original type and limit	Largest value now	Verdict
Bills and totals in paise	`long`, `long long`	94,668,000 paise per household-year, 7.57×10^{13} for the district	`long long` is sufficient

A.3 Which constants can no longer be fixed with `#define`

- Cannot stay as `#define`, because the board must change them without recompiling: `FIXED_CHARGE_PAISE`, `SLAB1_UNITS`, `SLAB2_UNITS`, `RATE1_PAISE`, `RATE2_PAISE`, `RATE3_PAISE`, and the **new** peak values (start hour, end hour, extra percent). They become fields of a `tariff_t` structure that is filled from a file.
- Can stay as `#define`, because they are facts that never change: `WH_PER_KWH` (1,000), `READINGS_PER_DAY` (96), `READINGS_PER_YEAR` (35,040), `PERCENT_BASE` (100), `MAX_INTERVAL_WH` (5,000), `MAX_HOUSEHOLD_NO` (800,000).

Evidence for A: tables A.1 to A.3.

B. EVALUATION OF THE ORIGINAL DESIGN (2 marks)

B.1 Calculation of the largest values

#	Quantity	Calculation	Result	Bits needed
1	Readings in a 31-day month	31 × 96	2,976	
2	Readings in a year	365 × 96	35,040	
3	Peak readings a day and in a year	4 hours × 4 = 16; 16 × 365	16 and 5,840	
4	Largest reading	20,000 W × 0.25 h	5,000 Wh	13
5	Largest energy in a 31-day month	2,976 × 5,000	14,880,000 Wh (14,880 kWh)	24
6	Largest energy in a year, one household	35,040 × 5,000	**175,200,000 Wh** (175,200 kWh)	**28** (2^{28} = 268,435,456)
7	Typical energy in a year, one household (150 kWh a month)	150 × 12 × 1,000	1,800,000 Wh	21
8	Largest energy, whole district, one year	800,000 × 175,200,000	**140,160,000,000,000 Wh** (1.4016×10^{14})	**47** ($2^{47} = 1.407 \times 10^{14}$)
9	Typical district energy, one year	800,000 × 1,800,000	1,440,000,000,000 Wh	41
10	Number of readings, whole district, one year	800,000 × 35,040	**28,032,000,000**	**35**
11	Unrounded charge of one reading (Wh × paise per kWh × percent)	5,000 × 450 × 120	270,000,000	29
12	Unrounded charge of one household-month	2,976 × 270,000,000	**803,520,000,000**	**40**
13	Unrounded charge of one household-year	35,040 × 270,000,000	9,460,800,000,000	44
14	Highest bill of one household-year (upper bound: every unit at Rs 4.50 + 20%, no slab discounts)	175,200 kWh × 450 × 1.2 = 94,608,000, plus 12 × 5,000	**94,668,000 paise** (Rs 946,680)	27
15	Highest district bill of a year (upper bound)	800,000 × 94,668,000	**75,734,400,000,000 paise** (7.57×10^{13})	47

The safe limits of the validation in my program (rates at most 100,000 paise a kWh, extra at most 1,000%) give at most 5,000 × 100,000 × 1,100 = 5.5×10^{11} for one reading and 1.64×10^{15} for a month, which is still 5,600 times below the `long long` maximum.

B.2 Evaluation of each part of the original design

Part of the original design	Still appropriate?	Calculation or evidence
Money in **integer paise**	**Yes, retained**	The reason for it (no floating-point errors, Tasks 2, 7, 10) is even stronger, because the unrounded charges are large and fractional
Part of the original design	**Still appropriate?**	**Calculation or evidence**
Rounding rule: half up, once per charge (Task 7)	**Yes, and now it is really used**	Fractions of a paisa now occur; for example 607.5 paise in the sample of section 4
`meter_t` (16-bit `unsigned short`) as the reading type	**No**	Largest year 175,200,000 Wh is 2,673 times the 65,535 limit. A 2 kW load (500 Wh a reading) would overflow it after only 132 readings (33 hours). A single reading (at most 5,000) would fit, but sums would not
`unsigned int` for units and energy	**No** for district totals	One household-year (175,200,000) fits in 32 bits, but the district-year (1.4016×10^{14}) is 32,000 times too large. Even the typical district-year (1.44×10^{12}) does not fit
`long` for bills and charges	**Not safe**	One household-month charge before rounding is 803,520,000,000, which overflows a 32-bit `long` (Windows)
`long long` for totals	**Yes, retained**	Largest value 7.57×10^{13} is about 121,800 times below its limit
Counters of readings	**No** if `unsigned int`	28,032,000,000 readings is more than 4,294,967,295; needs `unsigned long long`
`#define` for tariff constants	**No**	The board needs changes without recompiling (A.3)
`#define` for physical constants	**Yes, retained**	They never change
Roll-over arithmetic `(meter_t)(current - previous)`	**Not needed**	Readings are interval values. The check is replaced by a per-reading limit of 5,000 Wh
10,000-units-a-month check	**Replaced**	A 20 kW supply can use 14,880 kWh a month, so the old limit is not suitable. The program checks each reading against 5,000 Wh
Macros / `static inline`	**Retained as** `static inline`	The helper functions (`round_div`, `is_peak`, `days_in_month`) are `static inline`, with no macro side effects
Keeping all readings in memory	**Not possible**	28,032,000,000 readings at 2 bytes each is 56,064,000,000 bytes (56 GB). Readings must be processed as a stream

B.3 Measured proof (program `old_design_fails.c`, compiled with no warnings). It put the largest values into the original types:

```
1. one household, one year, maximum: 175200000 Wh
   stored in meter_t (16 bit)        : 22272   (correct value is 175200000)
stored in unsigned int (32 bit)   : 175200000
2. whole district, one year, maximum: 140160000000000 Wh   stored in
   unsigned int (32 bit)   : 2332229632 3. readings per year, whole
   district : 28032000000    stored in unsigned int (32 bit)   :
   2262196224 4. one household-month, scaled charge: 803520000000
   stored in unsigned int (32 bit)   : 361115648
5. a 2 kW load gives 500 Wh per reading; meter_t wraps after 132 readings (33.0
   hours)
6. storing every reading of the district as 2-byte values: 56064000000 bytes
   sizeof: short 2, int 4, long 8, long long 8
```

The one-year energy of one household, 175,200,000 Wh, became **22,272** in a `meter_t`; the district energy 140,160,000,000,000 Wh became 2,332,229,632 in an `unsigned int`; the 28,032,000,000 readings became 2,262,196,224; and the unrounded monthly charge 803,520,000,000 became 361,115,648. Every one is a wrong number with no warning at run time.

Evidence for B: tables B.1 and B.2 and the measured output in B.3.

C. ADAPTATION OR REDESIGN (2 marks)

C.1 The revised design

Design decision	Revised design
Money	Integer paise in `long long`, rounded half up once per household-month (the unrounded charge is a `long long` that counts Wh × paise per kWh × percent)
Design decision	**Revised design**
Energy	`long long` in Wh; kWh is shown by dividing by 1,000 with `%lld.%03lld`
Readings counter	`unsigned long long`
Tariff	`tariff_t` (9 `long long` fields) loaded from `tariff.cfg` by `load_tariff()`, with a range check for each key, and a check for missing and unknown keys
Peak-hour rule	`is_peak(hour)` is true from `peak_start_hour` (inclusive) to `peak_end_hour` (exclusive). The price percentage of a reading is 100 + `peak_extra_percent` in peak periods
Slab handling	`add_reading()` fills the slabs in time order. A reading that crosses a limit is split, so a slab limit is exact to 1 Wh
Rounding	`energy_paise = round_div(sum of Wh × rate × percent, 1,000 × 100)` is computed once per household-month
Input	One reading per line, read with `fgets` and `sscanf` (return value 6 required). One line at a time, so memory use does not grow with the data
Validation	Date and time ranges, minute 0/15/30/45, reading 0 to 5,000 Wh, records sorted by household and month
Output modes	`months` (every monthly bill), `years` (yearly energy and bill) and `totals` (only the totals)

How the money is calculated. For each reading, the energy is Wh and the price is `rate` paise for each kWh (that is, for 1,000 Wh) times `percent` / 100. So the charge in paise is Wh × rate × percent ÷ 100,000. Dividing every reading separately would lose fractions of a paisa, so the **numerator** Wh × rate × percent is added over the month as a whole number, and **one** division with half-up rounding is done at the end of the month.

C.2 Sample configuration file (`tariff.cfg`, 711 bytes)

```
# Grama Jyothi tariff configuration. All money is in paise.
# Edit this file to revise the tariff. The program does not need recompiling.

fixed_charge_paise = 5000      # Rs 50 per month slab1_units =
100              # first 100 kWh per month slab2_units = 200
# next 100 kWh (101 to 200) rate1_paise = 0                #
slab 1: free rate2_paise = 225              # slab 2: Rs 2.25
per kWh rate3_paise = 450              # slab 3: Rs 4.50 per kWh
above 200

peak_start_hour = 18           # peak starts at 18:00 (readings starting 18:00 or later)
peak_end_hour = 22             # peak ends at 22:00 (reading starting 22:00 is off-peak)
peak_extra_percent = 20        # peak units cost 20% more
```

C.3 Complete revised C program (`smart_billing.c`, 285 lines)

```
/* smart_billing.c - time-of-day billing for 15-minute smart-meter readings (in Wh).
   Compile: gcc -std=c11 -Wall -Wextra -o smart_bill smart_billing.c
   Run:     ./smart_bill tariff.cfg [months|years|totals] < readings.txt    Input
line: household month day hour minute wh   (sorted by household, then month) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ---- Fixed facts of the problem: #define is the right tool ---- */
#define READINGS_PER_DAY   96LL      /* 24 hours x 4 readings per hour            */
#define READINGS_PER_YEAR  35040LL   /* 365 days x 96                             */
#define WH_PER_KWH         1000LL    /* 1 unit = 1 kWh = 1000 Wh                  */
#define PERCENT_BASE       100LL #define MAX_INTERVAL_WH    5000LL    /* 20 kW for
15 minutes = 5 kWh = 5000 Wh    */
```

```
#define MAX_HOUSEHOLD_NO   800000LL
#define LINE_SIZE          128

/* ---- Tariff values: NOT compiled in, loaded from the configuration file ---- */
typedef struct {
    long long fixed_charge_paise;    /* per month
*/     long long slab1_units;           /* upper limit of slab 1, in kWh per month
*/     long long slab2_units;           /* upper limit of slab 2, in kWh per month
*/     long long rate1_paise;           /* paise per kWh in each slab
*/     long long rate2_paise;     long long rate3_paise;
    long long peak_start_hour;       /* peak starts at this hour (inclusive)
*/     long long peak_end_hour;         /* peak ends at this hour (exclusive)
*/     long long peak_extra_percent;    /* e.g. 20 means 20% more
*/ } tariff_t; typedef struct {
    const char *key;
long long  *field;
long long   min;
long long   max;
int         found; }
cfg_item_t;

/* Round half up, in integer arithmetic (values are never negative) */
static inline long long round_div(long long numerator, long long
denominator)
{
    return (numerator + denominator / 2) /
denominator; }

static inline int days_in_month(int month)
{
    static const int days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
return days[month - 1];                 /* a non-leap year: 365 days */ }

static inline int is_peak(const tariff_t *t, int hour)
{
    return hour >= t->peak_start_hour && hour < t-
>peak_end_hour; }

/* Reads "key = value" lines. Returns 1 if the file is complete and valid, else 0.
*/ static int load_tariff(const char *filename, tariff_t *t)
{
    FILE *fp = fopen(filename, "r");
if (fp == NULL) {
        fprintf(stderr, "Cannot open tariff file '%s'\n", filename);
return 0;
    }
    cfg_item_t items[] = {
        {"fixed_charge_paise",  &t->fixed_charge_paise, 0, 10000000, 0},
        {"slab1_units",         &t->slab1_units,        0, 100000,   0},
        {"slab2_units",         &t->slab2_units,        0, 100000,   0},
        {"rate1_paise",         &t->rate1_paise,        0, 100000,   0},
        {"rate2_paise",         &t->rate2_paise,        0, 100000,   0},
        {"rate3_paise",         &t->rate3_paise,        0, 100000,   0},
        {"peak_start_hour",     &t->peak_start_hour,    0, 23,       0},
        {"peak_end_hour",       &t->peak_end_hour,      1, 24,       0},
        {"peak_extra_percent",  &t->peak_extra_percent, 0, 1000,     0}
    };
    int count = (int)(sizeof items / sizeof
items[0]);     char line[LINE_SIZE];     int line_no
= 0, ok = 1;

    while (fgets(line, sizeof line, fp) != NULL)
{         char key[64];         long long value;
int matched = 0;
        char *p = line + strspn(line, " \t");
line_no++;
        if (*p == '#' || *p == '\n' || *p == '\r' || *p == '\0') {
            continue;                                   /* comment or blank line */
        }
        if (sscanf(p, "%63[A-Za-z0-9_] = %lld", key, &value) != 2) {
            fprintf(stderr, "Tariff file line %d: expected  key = number\n",
line_no);             ok = 0;             continue;
        }
        for (int i = 0; i < count; i++) {
if (strcmp(key, items[i].key) == 0) {
matched = 1;
```

```
                if (value < items[i].min || value > items[i].max) {
                    fprintf(stderr, "Tariff file line %d: %s must be %lld to
%lld\n",                                line_no, key, items[i].min, items[i].max);
ok = 0;                 } else {
                    *items[i].field = value;
items[i].found = 1;
                }
            }
        }         if
(!matched) {
            fprintf(stderr, "Tariff file line %d: unknown key '%s'\n", line_no,
key);              ok = 0;
        }
    }
    fclose(fp);
    for (int i = 0; i < count; i++) {
if (!items[i].found) {
            fprintf(stderr, "Tariff file: missing value for %s\n", items[i].key);
ok = 0;
        }
    }
    if (ok && t->slab1_units > t->slab2_units) {
        fprintf(stderr, "Tariff file: slab1_units must not exceed slab2_units\n");
ok = 0;
    }
    if (ok && t->peak_start_hour >= t->peak_end_hour) {
        fprintf(stderr, "Tariff file: peak_start_hour must be before peak_end_hour\n");
ok = 0;
    }
return ok; }

/* ---- Running totals for one household-month ---- */
typedef struct {
    long long household;           /* 0 means "no month open yet"
*/     int       month;     long long readings;
    long long month_wh;            /* energy so far this month, in Wh           */
long long peak_wh;
    long long numerator;           /* sum of (Wh x rate x percent), unrounded
*/ } month_state_t;

/* Adds one reading. Slabs are filled in time order, at Wh precision, so a
reading that crosses a slab limit is split between the two slabs. */
static void add_reading(month_state_t *m, const tariff_t *t, int hour, long long wh)
{
    long long percent = PERCENT_BASE + (is_peak(t, hour) ? t->peak_extra_percent :
0);     long long limit1 = t->slab1_units * WH_PER_KWH;     long long limit2 = t-
>slab2_units * WH_PER_KWH;     long long remaining = wh;

    while (remaining > 0) {
long long room, rate, take;
if (m->month_wh < limit1) {
            room = limit1 - m->month_wh;  rate = t->rate1_paise;
        } else if (m->month_wh < limit2) {
            room = limit2 - m->month_wh;  rate = t->rate2_paise;
        } else {
            room = remaining;             rate = t->rate3_paise;
        }
        take = (remaining < room) ? remaining :
room;         m->numerator += take * rate *
percent;         m->month_wh  += take;
remaining    -= take;
    }
    if (is_peak(t, hour)) {
m->peak_wh += wh;
    }
    m->readings++;
}

static void print_kwh(long long wh)
{
    printf("%lld.%03lld", wh / WH_PER_KWH, wh %
WH_PER_KWH); }

int main(int argc, char *argv[])
{
    tariff_t tariff;
```

```
    int mode = 2;                               /* 2 = months, 1 = years, 0 = totals
*/     char line[LINE_SIZE];     long long line_no = 0;

    month_state_t cur = {0, 0, 0, 0, 0, 0};
    long long year_wh = 0, year_peak_wh = 0, year_paise = 0, year_readings = 0;     long
long grand_paise = 0, grand_wh = 0, grand_peak_wh = 0, households = 0;     unsigned long
long accepted = 0, rejected = 0;     /* up to 2.8e10 readings per year */

    if (argc < 2 || !load_tariff(argv[1], &tariff)) {
        fprintf(stderr, "Usage: %s tariff.cfg [months|years|totals] < readings.txt\n", argv[0]);
return 1;
    }     if (argc
>= 3) {         if
(strcmp(argv[2],
"years") == 0)
mode = 1;
else if
(strcmp(argv[2],
"totals") == 0)
mode = 0;
else if
(strcmp(argv[2],
"months") != 0) {
fprintf(stderr,
"Unknown mode
'%s'\n", argv[2]);
return 1;
        }
    }

    for (;;) {
        long long household, wh;
int month, day, hour, minute;
        int have_line = (fgets(line, sizeof line, stdin) != NULL);

        if (have_line) {
line_no++;
            if (line[strspn(line, " \t\r\n")] == '\0') {
                continue;                                   /* blank line */
            }
            if (sscanf(line, " %lld %d %d %d %d %lld",
                       &household, &month, &day, &hour, &minute, &wh) != 6) {
                printf("Line %lld: REJECTED - invalid input (expected 6 numbers)\n", line_no);
rejected++;  continue;
            }
            if (household < 1 || household > MAX_HOUSEHOLD_NO || month < 1 || month > 12
||                day < 1 || day > days_in_month(month) || hour < 0 || hour > 23 ||
(minute != 0 && minute != 15 && minute != 30 && minute != 45)) {
                printf("Line %lld: REJECTED - household, date or time out of range\n", line_no);
rejected++;  continue;
            }
            if (wh < 0 || wh > MAX_INTERVAL_WH) {
                printf("Line %lld: REJECTED - reading must be 0 to %lld Wh\n",
line_no, MAX_INTERVAL_WH);                 rejected++;  continue;
            }
            if (cur.month != 0 && (household < cur.household ||
                (household == cur.household && month < cur.month))) {
                printf("Line %lld: REJECTED - records are not sorted by household and month\n",
line_no);                  rejected++;  continue;
            }
        }

        /* A new month (or the end of the input) closes the month that was open */
        if (cur.month != 0 && (!have_line || household != cur.household || month != cur.month)) {
            /* the fractional paise are rounded ONCE, here */
            long long energy = round_div(cur.numerator, WH_PER_KWH *
PERCENT_BASE);             long long total = tariff.fixed_charge_paise + energy;
if (mode == 2) {
                printf("Household %lld month %02d: readings %lld of %lld, energy
",                      cur.household, cur.month, cur.readings,
days_in_month(cur.month) * READINGS_PER_DAY);
print_kwh(cur.month_wh);                 printf(" kWh (peak ");
print_kwh(cur.peak_wh);
                printf(" kWh), fixed Rs %lld.%02lld, energy charge Rs %lld.%02lld, "
                       "total Rs %lld.%02lld\n",
                       tariff.fixed_charge_paise / 100, tariff.fixed_charge_paise % 100,
energy / 100, energy % 100, total / 100, total % 100);
            }
```

```
            year_wh += cur.month_wh;  year_peak_wh += cur.peak_wh;
year_paise += total;      year_readings += cur.readings;
            grand_paise += total;     grand_wh += cur.month_wh;   grand_peak_wh += cur.peak_wh;
            /* A new household (or the end of the input) closes the year
*/             if (!have_line || household != cur.household) {
if (mode >= 1) {
                    printf("Household %lld year: readings %lld of %lld, energy ",
cur.household, year_readings, READINGS_PER_YEAR);
print_kwh(year_wh);
                    printf(" kWh, yearly bill Rs %lld.%02lld\n",
year_paise / 100, year_paise % 100);
                }
households++;
                year_wh = year_peak_wh = year_paise = year_readings = 0;
            }
            memset(&cur, 0, sizeof cur);
        }
        if (!have_line) {
break;
        }
        if (cur.month == 0) {
cur.household = household;
cur.month = month;
        }
        add_reading(&cur, &tariff, hour,
wh);         accepted++;     }

    printf("Households billed: %lld, readings accepted: %llu, rejected:
%llu\n",            households, accepted, rejected);     printf("Total energy:
");     print_kwh(grand_wh);     printf(" kWh (peak ");
print_kwh(grand_peak_wh);
    printf(" kWh), grand total: Rs %lld.%02lld\n", grand_paise / 100, grand_paise %
100);     return 0; }
```

Compile and run:

- `gcc -std=c11 -Wall -Wextra -o smart_bill smart_billing.c` (no warnings)
- `./smart_bill tariff.cfg months < readings.txt` (use `years` or `totals` for shorter output)
- For large data, compile with `-O2`.

C.4 Sample input and output (small case, can be checked by hand)

For this example the test file `tariff_small.cfg` has the same keys as `tariff.cfg` but small slabs: `slab1_units = 1`, `slab2_units = 2` (everything else is as in the real tariff), so that a few readings cross the slab limits.

Input `sample_small.txt` (`household month day hour minute Wh`):

```
7 1 1 17 45 600
7 1 1 18 0 600
7 1 1 18 15 1000
7 1 1 22 0 10
7 1 1 22 15 500
7 1 2 18 20 100
7 1 2 abc 0 100
7 1 2 19 0 6000
7 2 1 18 0 1500
8 1 1 12 0 2500
7 3 1 0 0 100
```

Command: `./smart_bill tariff_small.cfg months < sample_small.txt`. Output (**produced by running it; long lines are wrapped to fit this page**):

```
Line 6: REJECTED - household, date or time out of range
Line 7: REJECTED - invalid input (expected 6 numbers)
Line 8: REJECTED - reading must be 0 to 5000 Wh
Household 7 month 01: readings 5 of 2976, energy 2.710 kWh (peak 1.600 kWh), fixed Rs 50.00,
energy charge Rs 6.08, total Rs 56.08
Household 7 month 02: readings 1 of 2688, energy 1.500 kWh (peak 1.500 kWh), fixed Rs 50.00,
energy charge Rs 1.35, total Rs 51.35
Household 7 year: readings 6 of 35040, energy 4.210 kWh, yearly bill Rs 107.43
Line 11: REJECTED - records are not sorted by household and month
Household 8 month 01: readings 1 of 2976, energy 2.500 kWh (peak 0.000 kWh), fixed Rs 50.00,
energy charge Rs 4.50, total Rs 54.50
Household 8 year: readings 1 of 35040, energy 2.500 kWh, yearly bill Rs 54.50
Households billed: 2, readings accepted: 7, rejected: 4
Total energy: 6.710 kWh (peak 3.100 kWh), grand total: Rs 161.93
```

Explanation of the output (hand calculation for household 7, month 1). Slab limits are 1,000 Wh and 2,000 Wh. The scaled charge is Wh × rate × percent.

No.	Time	Peak?	Wh	Month Wh before	Split between slabs	Wh × rate × percent
1	17:45	No	600	0	600 in slab 1 (rate 0)	0
2	18:00	Yes	600	600	400 in slab 1 (rate 0), 200 in slab 2	200 × 225 × 120 = 5,400,000
3	18:15	Yes	1,000	1,200	800 in slab 2, 200 in slab 3	800 × 225 × 120 + 200 × 450 × 120 = 21,600,000 + 10,800,000 = 32,400,000
4	22:00	No	10	2,200	10 in slab 3	10 × 450 × 100 = 450,000
5	22:15	No	500	2,210	500 in slab 3	500 × 450 × 100 = 22,500,000
No.	**Time**	**Peak?**	**Wh**	**Month Wh before**	**Split between slabs**	**Wh × rate × percent**
Sum			2,710			5,400,000 + 32,400,000 + 450,000 + 22,500,000 = **60,750,000**

Energy charge = 60,750,000 / 100,000 = **607.5 paise**, which rounds half up to **608 paise = Rs 6.08**. Bill = 5,000 + 608 = 5,608 paise = **Rs 56.08**, as the program printed. Energy used is 2.710 kWh, of which 600 + 1,000 = 1.600 kWh in peak periods (readings 2 and 3; the 17:45 and 22:00 readings are off-peak).

The other lines: month 2 of household 7 has 1,500 Wh at 18:00 (peak): 1,000 Wh are free and 500 Wh are in slab 2, 500 × 225 × 120 = 13,500,000, so 135 paise, Rs 51.35. Household 8 uses 2,500 Wh at 12:00 (off-peak): 1,000 free, 1,000 × 225 × 100 = 22,500,000 and 500 × 450 × 100 = 22,500,000, a total of 45,000,000, so 450 paise, Rs 54.50. The yearly line of household 7 is 5,608 + 5,135 = 10,743 paise = Rs 107.43. The grand total is 10,743 + 5,450 = 16,193 paise = **Rs 161.93**.

The four rejected lines are line 6 (minute 20 is not 0, 15, 30 or 45), line 7 (letters in the hour field, so `sscanf` did not return 6), line 8 (6,000 Wh is more than 5,000) and line 11 (household 7 after household 8, which is out of order). The reading counts show "5 of 2976" because the sample has only a few readings, and the program reports the expected number.

C.5 Sample output for a full year (5 households with 35,040 generated readings each; command `./smart_bill tariff.cfg years < year5.txt`):

```
Household 1 year: readings 35040 of 35040, energy 1260.170 kWh, yearly bill Rs
763.83
Household 2 year: readings 35040 of 35040, energy 3562.351 kWh, yearly bill Rs
9284.66
Household 3 year: readings 35040 of 35040, energy 5625.751 kWh, yearly bill Rs 19454.40
Household 799999 year: readings 35040 of 35040, energy 8346.847 kWh, yearly bill Rs 32919.87
Household 800000 year: readings 35040 of 35040, energy 12039.548 kWh, yearly bill Rs 51134.83
Households billed: 5, readings accepted: 175200, rejected: 0
Total energy: 30834.667 kWh (peak 14634.438 kWh), grand total: Rs 113557.59
```

Part of the monthly output (`months` mode) for household 1, which shows the effect of the slabs (February stays inside the free slab, and January goes a little beyond it):

```
Household 1 month 01: readings 2976 of 2976, energy 106.778 kWh (peak 45.799 kWh),
fixed Rs 50.00, energy charge Rs 16.85, total Rs 66.85
Household 1 month 02: readings 2688 of 2688, energy 96.108 kWh (peak 39.910 kWh),
fixed Rs 50.00, energy charge Rs 0.00, total Rs 50.00
```

Evidence for C: the complete working code (C.3), the configuration file (C.2), and the sample runs (C.4 and C.5).

D. EVIDENCE-BASED DEFENCE (2 marks)

D.1 Testing and validation (all runs with the final `smart_bill`)

#	Test	Expected (calculated independently)	Actual	Result
1	330 kWh used off-peak, extra 0% (66 readings of 5,000 Wh)	Same as the original program of Task 5: 330 units = Rs 860.00	Rs 860.00	Pass
2	The same 330 kWh off-peak with the real tariff (20% extra)	No peak units, so Rs 860.00	Rs 860.00	Pass
3	330 kWh all in peak hours, real tariff	5,000 + 1.2 × 81,000 = 102,200 paise = Rs 1,022.00	Rs 1,022.00	Pass

4	No consumption (one reading of 0 Wh)	Fixed charge only, Rs 50.00	Rs 50.00	Pass
5	Peak boundaries: readings at 17:45, 18:00, 21:45 and 22:00 of 1,000 Wh each	Peak = 2.000 kWh (18:00 and 21:45)	peak 2.000 kWh of 4.000 kWh	Pass
6	Slab splitting and rounding by hand (section C.4)	Household 7 Rs 56.08 (607.5 paise rounds up), Rs 51.35, household 8 Rs 54.50	Rs 56.08, Rs 51.35, Rs 54.50	Pass
7	Invalid lines: minute 20, letters, 6,000 Wh, unsorted household	All four rejected, valid lines unaffected	4 rejected, 7 accepted	Pass
8	Full year, 5 households (175,200 readings), against an independent Python calculation with exact fractions	Same 60 monthly bills and 5 yearly totals	60 of 60 monthly bills and 5 of 5 yearly totals match	Pass
#	**Test**	**Expected (calculated independently)**	**Actual**	**Result**
9	**Largest possible data:** one household with 5,000 Wh in all 35,040 readings	175,200.000 kWh, yearly bill Rs 807,180.00 (Python reference)	175,200.000 kWh, Rs 807,180.00, no overflow	Pass
10	Wrong tariff files: missing file, missing key, misspelt key, negative rate, text instead of number	Program stops with a message	All five stopped (exit status 1)	Pass
11	**Tariff revision without recompiling:** rate 2 from 225 to 230 and extra from 20% to 25%, test 3 data	100,000 Wh × 230 × 125 / 100,000 = 28,750 paise plus 130,000 Wh × 450 × 125 / 100,000 = 73,125 paise, plus 5,000 = 106,875 paise = Rs 1,068.75	Rs 1,068.75	Pass
12	Speed: 50 households, 1,752,000 readings (29.5 MB)		0.49 s in `totals` mode with `-O2`	Measure d

The independent reference calculation (test 8) uses `Fraction` numbers in Python and finds the slab overlap by a different method (intervals of cumulative Wh), so it does not repeat the logic of the C program. The test 9 value, Rs 807,180.00, is exactly the same in the C program and in that reference.

D.2 Float against integer paise on the same year of data (program `float_check.c`). The same 5 household-years were billed with `float` and `double` rupees (adding the charge of each reading in floating point and not rounding the months), and compared with the integer program:

Household	Integer paise (monthly bills rounded once, the defined rule)	double rupees	float rupees
1	Rs 763.83	Rs 763.8351	Rs 763.84
2	Rs 9,284.66	Rs 9,284.6656	Rs 9,284.67
3	Rs 19,454.40	Rs 19,454.3916	Rs 19,454.39
799,999	Rs 32,919.87	Rs 32,919.8721	Rs 32,919.88
800,000	Rs 51,134.83	Rs 51,134.8443	**Rs 51,134.74**

Read the table carefully. The `double` column is the unrounded sum, so it differs from the integer column by less than one paisa, which is only the rounding rule (the integer design rounds **each month** once, as the written rule says). The `float` column shows genuine floating-point error, and it grows with the size of the total: for household 800,000 (the largest energy, 12,039 kWh) `float` is about 10 paise below the `double` sum (Rs 51,134.74 against Rs 51,134.8443) and 9 paise below the correct bill. For the small households the `float` error is hidden by the 2-decimal printing. The integer result is the sum of 12 monthly bills, each rounded once by the written rule, so it is repeatable and a customer can check it. A floating-point result depends on the order of additions and cannot follow a month-by-month rounding rule exactly.

D.3 The largest values against the limits of the chosen types

Quantity	Largest value	Chosen type and its limit	Margin
Energy of one household, year	1.752×10^8 Wh	`long long`, 9.22×10^{18}	5.3×10^{10} times

Energy of the district, year	1.4016×10^{14} Wh	`long long`	66,000 times
Readings of the district, year	2.8032×10^{10}	`unsigned long long`, 1.84×10^{19}	6.6×10^{8} times
Unrounded charge, household-month	8.04×10^{11}	`long long`	1.1×10^{7} times
District bills, year (upper bound)	7.57×10^{13} paise	`long long`	121,800 times

D.4 Trade-offs, and why the decision stands

Concern	Trade-off	Why the decision stands
Memory	`long long` is 8 bytes where 2 would do for one reading	The data is processed as a stream, so only a small state is kept: the tariff (72 bytes), one household-month (48 bytes on my system) and a 128-byte line buffer. Memory use does not grow with 28 billion readings
Speed	Text parsing of every reading, about 3.6 million readings a second (1,752,000 in 0.49 s)	28,032,000,000 readings take about 7,800 s (2.2 hours) on one core. Households are independent, so the file can be split by household over several cores, and the work is done once a month, not every second
Concern	**Trade-off**	**Why the decision stands**
Safety	A mistyped tariff file could give wrong bills	`load_tariff()` refuses a file with a missing, unknown, non-numeric or out-of-range value (test 10)
Interpretation of the 20% rule	Slabs are filled in time order, so the result depends on when the energy is used	This is stated as assumption 4, the code is short (`add_reading`), and the board can change the rule in one function. It reproduces the original bills exactly when the extra is 0% (test 1)
Assumed 20 kW limit	A larger supply would need a larger limit	`MAX_INTERVAL_WH` is one `#define`, and even a limit 100 times larger would keep the upper-bound district bills (7.57×10^{15} paise) about 1,200 times below the `long long` limit
Complexity	Time per reading is O(1) (at most 3 slab steps), so total time is O(R) for R readings. Memory is O(1)	A table lookup or sorting is not needed

D.5 Original design against the redesigned system

Property	Original design (Tasks 1-11)	Redesigned smart-meter system
Readings per household per year	12 (one a month)	35,040 (every 15 minutes)
Reading unit and type	kWh in a 16-bit `meter_t`	Wh in `long long`
Roll-over handling	`(meter_t)(current - previous)`	Not needed (interval values)
Tariff	Slabs, compiled in (`#define`)	Slabs and +20% peak, loaded from `tariff.cfg` at start-up
Tariff revision	Edit header, recompile, redistribute	Edit one text file
Money	Integer paise, `long` bills, `long long` totals	Integer paise in `long long` everywhere
Rounding	None needed (whole-paise rates)	Half up, once per household-month, on an integer numerator
Largest district total	3.55×10^{12} paise (10,000-unit limit)	7.57×10^{13} paise (upper bound)
Input handling	Whole file, one record per household	Streamed, 35,040 lines per household, sorted
Validation	Range, roll-over plausibility (10,000 units)	Date, time, reading (0 to 5,000 Wh), order, tariff file
Concepts kept		Integer paise, round once, validate before billing, `long long` totals, `static inline`, no warnings

D.6 Final decision. The original design is **modified**: (1) the old types are proven too small by calculation and by the measured wrong values in B.3 (22,272 instead of 175,200,000); (2) the integer-paise design is kept, because floating point drifted by about 10

paise on the largest household's yearly bill (D.2) and gave wrong totals in Tasks 2, 7 and 10, and because only integers can follow the month-by-month rounding rule exactly; (3) the tariff is moved to a configuration file, because a start-up read costs about 4 microseconds and the board needs changes without recompiling (Task 11, section 6); (4) all results were checked against an independent exact calculation (60 of 60 monthly bills) and at the largest possible data (Rs 807,180.00, with no overflow), and the program still gives the original bills when the new rules are switched off (test 1). The redesign is supported by the largest-value calculations (B.1), the tests (D.1), and the trade-offs (D.4).

Evidence for D: the 12 tests, the float comparison, the margin table and the trade-off table, which together connect the changed requirements, the largest values, the test results and the decision.

Evidence checklist for Task 12

- Screenshots to add: compilation with `-Wall -Wextra` (no warnings); the small sample run; the `years` output; the maximum-load run (Rs 807,180.00); the wrong tariff file messages; the tariff revision (Rs 1,022.00 to Rs 1,068.75).